

Web and Data Engineering

Lecture 04: HTTP und RESTful APIs

Prof. Dr. Michael Granitzer

Speaker notes

Diese Vorlesung behandelt HTTP als Kommunikationsprotokoll des Webs und REST als darauf aufbauenden Architekturstil für APIs (Application Programming Interfaces, also Programmierschnittstellen). Im Mittelpunkt stehen Nachrichtenaufbau, HTTP-Semantik, Caching, Sicherheit, Ressourcenmodellierung und der Lebenszyklus von APIs.



Ziel dieser Vorlesung

HTTP ist das Protokoll des Webs. REST nutzt dieses Protokoll konsequent als Grundlage für API-Design.

Diese Vorlesung verbindet beides: vom **Protokollverständnis über Caching und Sicherheit** bis zum **Entwurf robuster, evolvierbarer APIs**.

Leitfragen

- Warum ist HTTP mehr als Datentransport?
- Wie strukturieren Header HTTP-Nachrichten und ihre Repräsentationen?
- Wie steuern Header Caching, Sessions und Sicherheit?
- Was unterscheidet REST von beliebigem JSON-über-HTTP?
- Wie modelliert man Ressourcen, URIs und HTTP-Semantik für APIs?
- Wie entwickelt man APIs weiter, ohne Clients zu brechen?

Speaker notes

HTTP definiert, wie Clients und Server Nachrichten austauschen. REST nutzt diese Mechanismen konsequent für API-Design. Lernziel ist, HTTP-Nachrichten fachlich zu lesen, Header, Medienformate und Statuscodes korrekt zu interpretieren und APIs (Application Programming Interfaces) anhand von Ressourcen, Semantik und Evolvierbarkeit zu bewerten.



HTTP Grundlagen

Leitfrage: Wie können Browser und Server mit einem einfachen, textbasierten Protokoll so kommunizieren, dass das Web weltweit skalierbar bleibt?

HTTP ist ein standardisiertes Request-Response-Protokoll. Es trennt klar zwischen Client und Server und bildet die Grundlage für Web-Seiten, Web-APIs, Caching-Infrastruktur und viele Sicherheitsmechanismen.

Was sehen Sie hier?

Ein Nutzer ruft die Produktseite auf. Das Netzwerk-Tab zeigt diesen Austausch:

↑ REQUEST

```
GET /api/products/42 HTTP/1.1
Host: shop.example.com
Accept: application/json
Cookie: session=abc123
```

↓ RESPONSE

```
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: max-age=300
ETag: "f7e3a1b2"

{"id":42,"name":"Funkkopfhörer","price":79.99}
```

Aufgabe: Was ist Startzeile, Header, Body? Welche Information steckt in jedem Header?

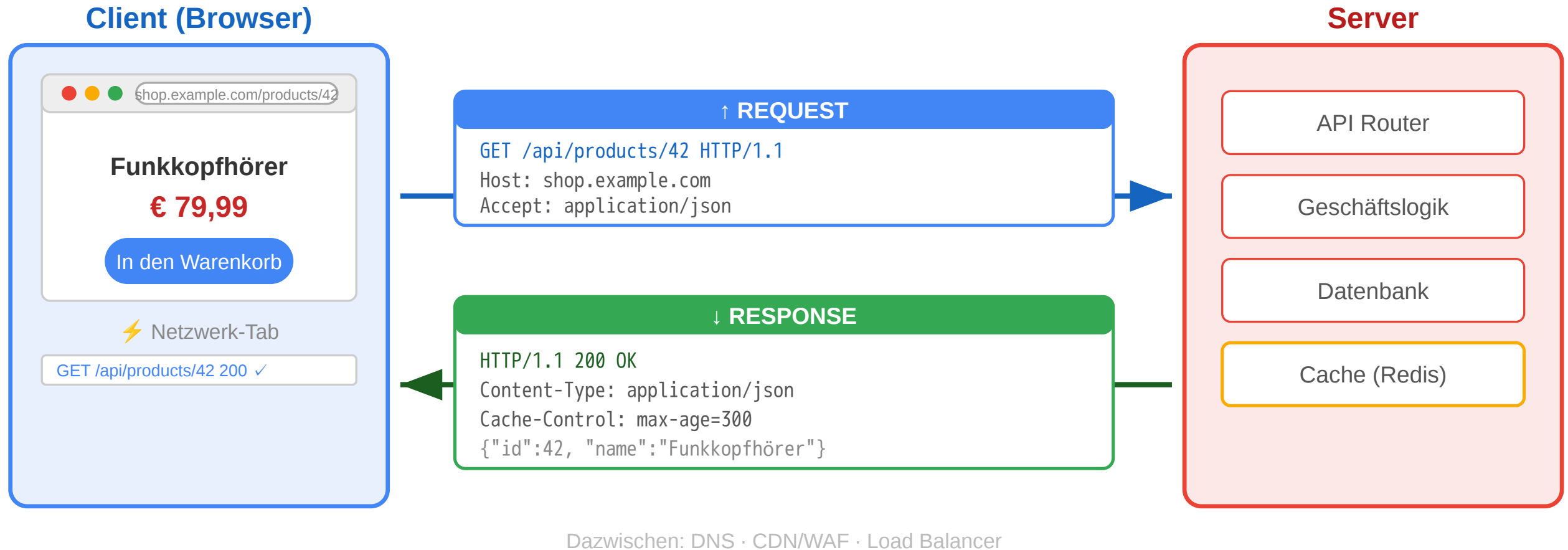
Teil	Request	Response
Startzeile	GET /api/products/42 HTTP/1.1	HTTP/1.1 200 OK
Header	Host, Accept, Cookie	Content-Type, Cache-Control, ETag
Body	kein Body bei GET	JSON-Daten des Produkts

Cache-Control: max-age=300
Daten 5 min gültig
ETag: "f7e3a1b2"
Fingerprint zur Validierung



Ein HTTP-Austausch besteht aus Startzeile, Headern und optionalem Body. Im Request beschreibt die Startzeile Methode, Ziel und Protokollversion. Im Response beschreibt die Startzeile Version, Statuscode und Statusmeldung. Header transportieren Metadaten wie akzeptierte Formate, Session-Informationen oder Cache-Regeln. Der Body enthält die eigentliche Repräsentation der Ressource, hier JSON (JavaScript Object Notation).

HTTP: Das Request-Response-Protokoll



HTTP arbeitet nach einem einfachen Muster: Der Client sendet einen Request an eine Adresse, der Server verarbeitet ihn und liefert einen Response. Diese Struktur ermöglicht eine lose Kopplung zwischen Browsern, Servern, Proxies, Caches und Gateways.

Von der Fetch API zum Protokoll

In Vorlesung 03 haben wir die **Client-Seite** gesehen:

```
const response = await fetch('/api/products/42', {
  headers: { 'Accept': 'application/json' }
});
```

Jetzt schauen wir auf die **Protokollseite**:

Fetch API	HTTP-Protokoll
fetch(url)	Browser erzeugt HTTP-Request mit Startzeile + Headern
{ method: 'POST' }	HTTP-Methode — definiert Absicht und Semantik
{ headers: {...} }	Request-Header — Metadaten für Server und Infrastruktur
response.status	Statuscode — standardisiertes Ergebnis
response.json()	Response Body parsen — Repräsentation der Ressource

Die Fetch API ist eine **JavaScript-Abstraktion über HTTP**. Wer HTTP versteht, versteht auch, warum fetch so funktioniert wie es funktioniert — und kann Netzwerkprobleme im DevTools-Netzwerk-Tab diagnostizieren.

Die Fetch API ist eine Programmierschnittstelle im Browser, die HTTP-Nachrichten erzeugt und Responses kapselt. `method`, `headers` und `Body` in JavaScript entsprechen direkt Bestandteilen einer HTTP-Nachricht. Wer HTTP versteht, kann Netzwerkverhalten unabhängig vom verwendeten Framework analysieren.

HTTP-Designprinzipien

Prinzip	Bedeutung	Konsequenz
Ressourcenorientiert	Jede Information hat eine Adresse (URI)	Einheitliche Adressierung weltweit
Request-Response	Client fragt, Server antwortet	Klare Rollenverteilung
Zustandslos	Jeder Request ist unabhängig	Server muss keinen Kontext halten → skaliert
Erweiterbar	Header erlauben beliebige Metadaten	Caching, Auth, Content-Negotiation ohne Protokolländerung
Textbasiert	Nachrichten sind menschenlesbar (HTTP/1.1)	Einfach zu debuggen

Warum das skaliert

- Zustandslosigkeit erlaubt Load Balancing über beliebig viele Server
- Proxies und Caches können HTTP-Nachrichten ohne Anwendungswissen verarbeiten
- Klare Trennung von Transport und Anwendungslogik

URI (Uniform Resource Identifier)

`https://shop.example.com:443/api/products/42?lang=de#details`

Teil	Beispiel	Bedeutung
Scheme	https	URI-Schema vor ;; bestimmt, wie die URI interpretiert wird
Host	shop.example.com	DNS-Name des Servers
Port	443	TCP-Port; bei HTTPS implizit 443
Pfad	/api/products/42	Adresse der Ressource auf dem Host
Query	?lang=de	Parameter für Auswahl, Filter, Darstellung

HTTP ist ressourcenorientiert, zustandslos und durch Header erweiterbar. Zustandslosigkeit bedeutet, dass jeder Request für sich interpretierbar sein muss. Dadurch werden Lastverteilung, Zwischenspeicherung und horizontale Skalierung erleichtert. URIs (Uniform Resource Identifiers) geben Ressourcen stabile Adressen.

HTTP-Methoden als semantischer Vertrag

Methode	Absicht	Sicher?	Idempotent?	Online-Shop-Beispiel
GET	Ressource lesen	ja	ja	Produktseite anzeigen
POST	Neue Ressource oder Verarbeitung	nein	nein	Bestellung aufgeben
PUT	Ressource vollständig ersetzen	nein	ja	Profil aktualisieren
PATCH	Ressource teilweise ändern	nein	nein*	Adresse ändern
DELETE	Ressource entfernen	nein	ja	Konto löschen
HEAD	Nur Header, kein Body	ja	ja	Existenzprüfung
OPTIONS	Erlaubte Methoden abfragen	ja	ja	CORS Preflight

Sicher (Safe): Methode verändert keine Ressource — kann ohne Nebenwirkungen wiederholt werden. CDNs cachen nur safe Methoden.

Idempotent: Mehrfache Ausführung hat denselben Effekt wie einmal — wichtig für Retry-Logik in Load Balancern und API-Gateways.

Achtung: Die Unterscheidung ist eine **Konvention**, keine technische Durchsetzung. Jeder Server kann jede Methode für jede semantische Bedeutung nutzen. Die Tabelle beschreibt die **übliche** Interpretation.

HTTP-Methoden kodieren Absichten. GET liest, POST erzeugt oder stößt Verarbeitung an, PUT ersetzt, PATCH ändert teilweise, DELETE entfernt. Safe bedeutet: keine Änderung am fachlichen Zustand. Idempotent bedeutet: Mehrfachausführung hat denselben Effekt wie einmalige Ausführung.

HTTP-Methoden: Safe und Idempotent in der Praxis

Szenario: Ein Nutzer klickt zweimal schnell auf „Jetzt kaufen“. Netzwerk-Lag verzögert den ersten Request, beide kommen beim Server an.

Methode	Effekt beim Doppel-Request	Warum?
POST /api/orders	Zwei Bestellungen entstehen	Nicht idempotent — jeder Aufruf erzeugt neue Ressource
PUT /api/cart/item/42	Eine Änderung — zweiter Call ohne Effekt	Idempotent — gleiches Ergebnis egal wie oft

Konsequenz: Load Balancer und API-Gateways können idempotente Requests bei Timeout sicher wiederholen. CDNs cachen nur **safe** Methoden (GET, HEAD), weil diese die Ressource nie verändern.

Die Unterscheidung zwischen safe und idempotent hat direkte Auswirkungen auf Betrieb und Fehlertoleranz. Idempotente Requests können bei Netzwerkproblemen sicher wiederholt werden. Nicht-idempotente POST-Requests brauchen zusätzliche Schutzmechanismen, damit Wiederholungen keine doppelten Effekte erzeugen.

HTTP-Statuscodes

Klasse	Bedeutung	Wichtige Codes
1xx	Information	101 Switching Protocols
2xx	Erfolg	200 OK, 201 Created, 204 No Content
3xx	Umleitung	301 Moved Permanently, 304 Not Modified
4xx	Client-Fehler	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Too Many Requests
5xx	Server-Fehler	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable

Was stimmt an dieser API-Antwort nicht?

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "success": false,
  "error": "Produkt nicht gefunden"
}
```

Problem: Statuscode lügt

- 200 OK signalisiert Erfolg
- Body enthält aber einen Fehler
- CDNs cachen die „Erfolg“-Antwort
- Monitoring zählt 200 als OK

Richtig: 404 Not Found als Statuscode, Fehlerdetails im Body.

Statuscodes sind maschinenlesbare Standardantworten. 2xx steht für Erfolg, 3xx für Umleitungen, 4xx für Probleme auf Client-Seite und 5xx für Probleme auf Server-Seite. Der Statuscode beschreibt die eigentliche Bedeutung des Ergebnisses; ein Fehler darf nicht nur im Body versteckt werden.

HTTP-Versionen im Vergleich

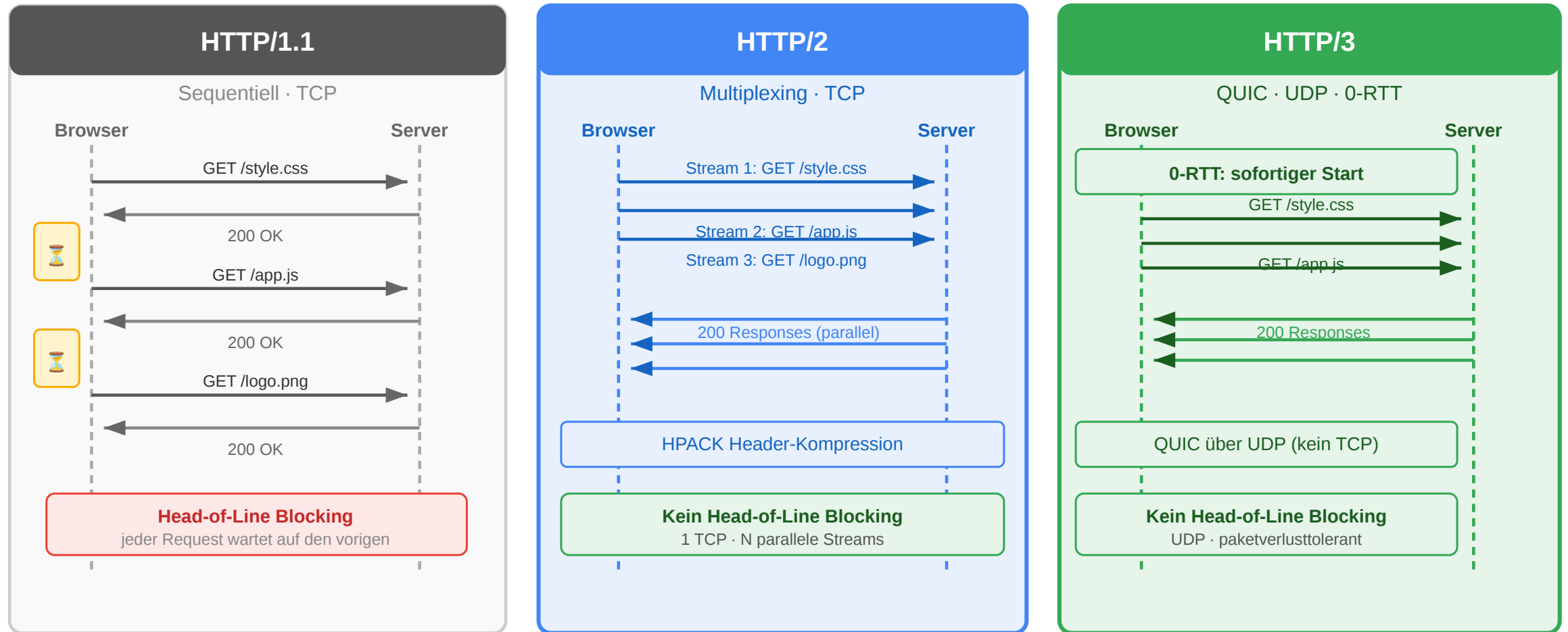
Version	Jahr	Kernverbesserung
HTTP/1.0	1996	Ein Request pro TCP-Verbindung
HTTP/1.1	1997	Persistent Connections, Pipelining
HTTP/2	2015	Multiplexing, Header-Kompression, binär
HTTP/3	2022	QUIC (UDP-basiert), schnellerer Aufbau

QUIC ist der Transport unter HTTP/3: Es läuft über UDP, integriert Verschlüsselung direkt und kann Verbindung plus TLS schneller aufbauen. Wichtig: QUIC ersetzt hier TCP, HTTP-Semantik wie Methoden, Statuscodes und Header bleibt erhalten.

HTTP ist nicht nur Datentransport — es ist das **Interaktionsmodell** des Webs. Methoden definieren Absichten, Statuscodes kommunizieren Ergebnisse, Header steuern Verhalten. Infrastruktur (CDN, Load Balancer, WAF) verlässt sich auf diese Semantik. Wer HTTP als Syntax behandelt, verliert die Vorteile des Protokolls.

HTTP/1.0 öffnete typischerweise pro Request eine neue TCP-Verbindung. HTTP/1.1 führte persistent connections ein. HTTP/2 multiplexiert mehrere Requests über eine Verbindung und komprimiert Header. HTTP/3 basiert auf QUIC (Quick UDP Internet Connections) über UDP (User Datagram Protocol) und reduziert Verbindungsaufbau und Transportlatenz.

HTTP-Versionen im Vergleich



Frage: Eine Produktseite lädt 60 Ressourcen (JS, CSS, Bilder). Warum dauert das mit HTTP/1.1 deutlich länger als mit HTTP/2 — obwohl die Bandbreite dieselbe ist?

HTTP Header und Repräsentationen

Leitfrage: Wie beschreiben HTTP-Header, welches Format gesendet wird, wie es interpretiert werden soll und welche Varianten einer Ressource ausgehandelt werden?

HTTP trennt nicht nur zwischen Request und Response, sondern auch zwischen fachlicher Ressource und ihrer konkreten Darstellung. Header-Felder liefern die dazugehörigen Metadaten, etwa Medienformat, Sprache, Kompression oder Cache-Regeln.

HTTP-Header im Überblick

Eine HTTP-Nachricht besteht aus:

1. Startzeile

Request: Methode + Ziel + Version

Response: Version + Statuscode + Statusmeldung

2. Header-Felder

Metadaten als Name: Wert

3. Leerzeile

Trennt Header und Body

4. Optionaler Body

Die eigentliche Repräsentation

```
GET /products/42 HTTP/1.1
```

```
Host: shop.example.com
```

```
Accept: application/json
```

Header-Felder stehen zwischen Startzeile und Body. Sie beschreiben nicht die Ressource selbst, sondern Zusatzinformationen zur Nachricht, etwa Zielhost, gewünschtes Format, Authentifizierung, Cache-Verhalten oder Cookie-Daten. Die Leerzeile ist technisch relevant, weil sie den Headerblock vom optionalen Body trennt. Erst durch diese Struktur können generische Werkzeuge wie Browser, Proxies oder Debugger Nachrichten systematisch lesen.

Wie sind HTTP-Header strukturiert?

Teil	Bedeutung	Beispiel
Feldname	Standardisierter Header-Name	Content-Type
Doppelpunkt	Trennt Name und Wert	:
Feldwert	Semantik des Headers	application/json

```
Content-Type: application/json; charset=utf-8  
Cache-Control: public, max-age=3600  
Authorization: Bearer eyJhbGciOi...
```

- Header-Namen sind **case-insensitive**
- Ein Feldwert kann aus **Token, Parametern oder Listen** bestehen
- Viele Header haben eine klar definierte **Syntax nach RFC**

Die allgemeine Form ist immer Feldname: Wert. Die genaue Struktur des Werts hängt vom Header ab: Media Types haben optionale Parameter wie charset, Cache-Control kombiniert mehrere Direktiven, Accept-Header können mehrere Werte mit Prioritäten enthalten.

Kategorien von Header-Feldern

Kategorie	Typische Header	Zweck
Request-Header	Host, Accept, Authorization, Cookie	Beschreiben den eingehenden Request
Response-Header	Server, Location, Set-Cookie	Beschreiben Eigenschaften der Antwort
Repräsentations-Header	Content-Type, Content-Length, Content-Encoding, Content-Language	Beschreiben den Body
Caching / Conditional	Cache-Control, ETag, If-None-Match, Last-Modified	Steuern Wiederverwendung und Validierung
Sicherheits-Header	Strict-Transport-Security, Content-Security-Policy	Erhöhen Sicherheitseigenschaften

Wichtig: Dieselbe HTTP-Ressource kann mehrere Repräsentationen haben. Die Header beschreiben, **welche Variante gerade übertragen wird**.

Diese Kategorien helfen beim Lesen realer HTTP-Nachrichten. Besonders wichtig für das Verständnis von REST ist die Trennung zwischen Ressourcenidentität durch URI (Uniform Resource Identifier) und Repräsentationsbeschreibung durch Header.

Typische Request- und Response-Header

Header	Typ	Kurz erklärt
Host	Request	Gibt an, für welchen Host der Request bestimmt ist; wichtig bei virtuellen Hosts
Authorization	Request	Überträgt Anmeldedaten oder Zugriffstoken
Location	Response	Verweist auf die URI einer neuen oder anderen Ressource
Server	Response	Kann Informationen über die Server-Software enthalten

```
POST /login HTTP/1.1
Host: shop.example.com
Authorization: Bearer eyJ...
```


Host ist seit HTTP/1.1 zentral, weil mehrere Websites auf derselben IP-Adresse betrieben werden können. Authorization überträgt Zugriffsinformationen, etwa ein Bearer-Token. Location wird oft bei Redirects oder 201 Created verwendet. Server ist für Clients meist nicht funktional nötig, taucht aber in realen Responses häufig auf.

Typische Header für Repräsentation und Kontrolle

Header	Kategorie	Kurz erklärt
Content-Length	Repräsentation	Nennt die Länge des Bodys in Bytes
Content-Encoding	Repräsentation	Gibt an, ob der Body z. B. mit gzip komprimiert wurde
Content-Security-Policy	Sicherheit	Beschränkt, welche Inhalte eine Seite laden oder ausführen darf
Strict-Transport-Security	Sicherheit	Erzwingt für eine Domain die Nutzung von HTTPS

Merksatz:

Ein Teil der Header beschreibt den **Body**, ein anderer Teil steuert **Verhalten** von Clients, Caches und Browsern.

Content-Length und Content-Encoding beschreiben Eigenschaften der übertragenen Repräsentation, die im Alltag leicht übersehen werden. Content-Security-Policy und Strict-Transport-Security sind typische Sicherheits-Header für Web-Anwendungen; letzterer wird oft als HSTS für HTTP Strict Transport Security abgekuerzt.

Medienrepräsentationen und MIME Types

Eine Ressource ist nicht gleich ihr Datenformat. Dieselbe Ressource kann als verschiedene **Medientypen** übertragen werden:

Medientyp	Bedeutung	Beispiel
text/html	HTML-Dokument	Produktseite im Browser
application/json	JSON-Daten	API-Response
text/css	Stylesheet	Gestaltung einer Seite
application/javascript	JavaScript-Code	Skriptdatei
image/png	PNG-Bild	Produktfoto
application/pdf	PDF-Dokument	Rechnung zum Download

Media Type = Typ/Subtyp

Beispiele: text/html, application/json, image/svg+xml

Der Medientyp beschreibt nicht die Ressource selbst, sondern die Form ihrer aktuellen Repräsentation.

MIME Type bedeutet ursprünglich Multipurpose Internet Mail Extensions; im Web spricht man meist gleichbedeutend von Media Types. Der Media Type sagt Client und Infrastruktur, wie der Body interpretiert werden soll. Ohne korrekten Content-Type weiß ein Browser oder API-Client nicht verlässlich, wie die empfangenen Bytes zu lesen sind. Ein Browser behandelt `text/html` anders als `image/png` oder `application/json`; genau deshalb ist der Medientyp Teil der HTTP-Semantik und nicht nur Dekoration.

Verschiedene Clients, verschiedene Repräsentationen

Nicht jeder Client braucht dieselbe Darstellung derselben Ressource:

Client-Modell	Typische Repräsentation	Wofür geeignet?
Server-rendererter Browser-Client	text/html	Der Server liefert direkt eine fertige HTML-Seite
JavaScript-Frontend / SPA	application/json	Das Frontend rendert die Oberfläche selbst
Mobile App	application/json	Die App verarbeitet Daten und baut die UI nativ
Download-Client	application/pdf, image/png	Datei- oder Medienaussgabe

Kernidee:

Die fachliche Ressource bleibt gleich, aber die **Repräsentation** wird an den Bedarf des Clients angepasst.

Die drei Client-Modelle nutzen dieselbe Ressource, aber unterschiedliche Repräsentationen: Ein server-renderter Browser-Client erwartet `text/html`, weil der Browser direkt anzeigt, was der Server liefert. Ein JavaScript-SPA oder eine mobile App erwartet `application/json`, weil sie die Darstellung selbst übernehmen — die Daten kommen als maschinenlesbare Struktur, die Rendering-Logik liegt im Client. Download-Clients (PDF-Export, Bilddienste) erwarten binäre MIME-Types. Der `Accept-Header` des HTTP-Requests teilt dem Server mit, welche Repräsentation der Client verarbeiten kann — Content Negotiation erlaubt dem Server, die passende Variante auszuliefern.

HTML vs. JSON als Repräsentation

HTML-Repräsentation

- Enthält Struktur und Inhalt für die direkte Anzeige im Browser
- Typisch für **serverseitig gerenderte** Anwendungen
- Media Type: text/html

JSON-Repräsentation

- Enthält strukturierte Daten statt fertiger Seiten
- Typisch für **APIs**, SPAs und mobile Clients
- Media Type: application/json

Wichtig:

HTML und JSON konkurrieren nicht zwingend. Beide können sinnvolle Repräsentationen **derselben Ressource** sein.

Serverseitig gerendertes HTML bedeutet, dass der Server die sichtbare Seite zusammensetzt. JSON ist dagegen vor allem eine Datenrepräsentation, die von Browser-JavaScript, mobilen Apps oder anderen Diensten weiterverarbeitet wird.

JSON kurz eingeführt

JSON steht für **JavaScript Object Notation**. Es ist ein leichtgewichtiges Textformat für strukturierte Daten.

```
{
  "id": 42,
  "name": "Funkkopfhörer",
  "price": 79.99,
  "in_stock": true,
  "tags": ["audio", "wireless"],
  "manufacturer": {
    "name": "Acme",
    "country": "DE"
  },
  "discount": null
}
```

JSON-Element	Beispiel
Objekt	{ "id": 42 }
Array	[1, 2, 3]
String	"name"
Zahl	79.99
Boolean	true, false
Null	null

JSON ist im Web so verbreitet, weil es kompakt, textbasiert und in fast allen Programmiersprachen leicht verarbeitbar ist. Für REST ist aber wichtig: JSON ist nur eine mögliche Repräsentation, nicht die Definition von REST.

Content Negotiation: Grundidee

Header	Richtung	Bedeutung
Content-Type	Request oder Response	Welches Format der Body hat
Accept	Request	Welche Formate der Client akzeptiert
Accept-Language	Request	Bevorzugte Sprache
Accept-Encoding	Request	Bevorzugte Kompression, z. B. gzip oder br
Content-Encoding	Response	Welche Kompression auf den Body angewendet wurde

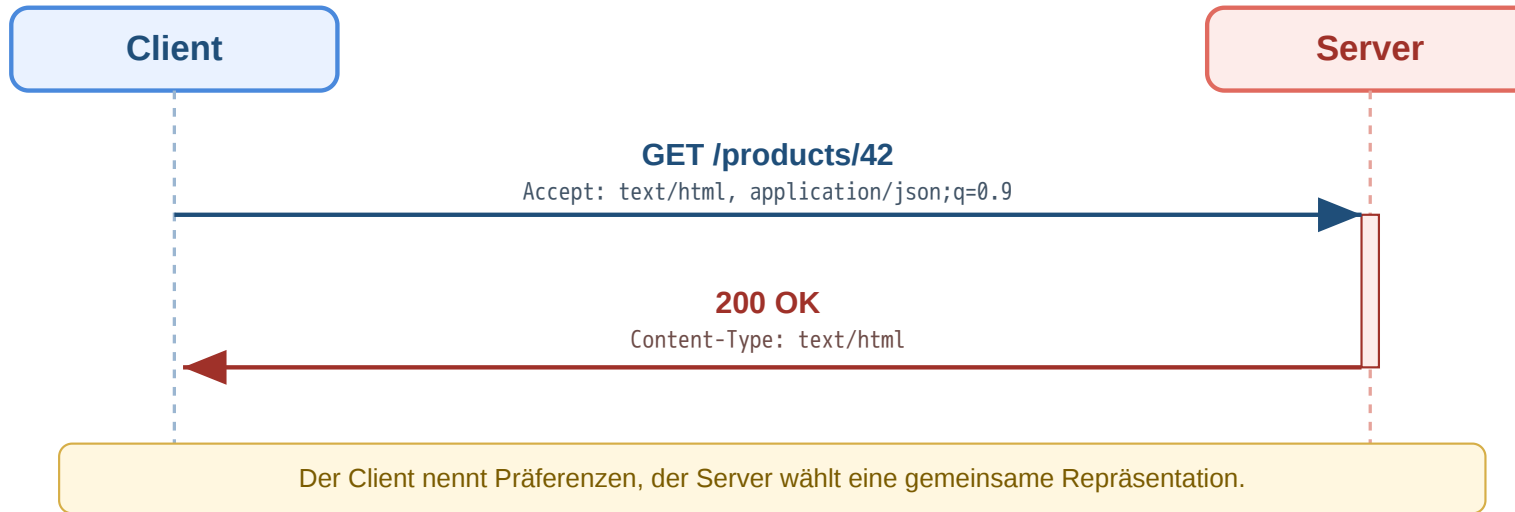
```
GET /products/42 HTTP/1.1
Accept: application/json
Accept-Language: de
Accept-Encoding: gzip, br
```

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Language: de
Content-Encoding: gzip
```

Content Negotiation bedeutet, dass der Client Präferenzen mitteilt und der Server eine passende Repräsentation wählt. Accept beschreibt das gewünschte Format, Content-Type das tatsächlich gelieferte Format. Dasselbe Muster gibt es auch für Sprache und Kompression.

Content Negotiation: Repräsentation auswählen

Ein Client kann mehrere Formate akzeptieren und Prioritäten angeben:



Teil	Bedeutung
text/html	Bevorzugte Repräsentation
application/json;q=0.9	Ebenfalls akzeptabel, aber etwas weniger bevorzugt
application/xml;q=0.5	Nur niedrige Priorität

q-Wert = Qualitätswert

Je höher der q-Wert, desto stärker die Präferenz des Clients.

Wenn kein gemeinsames Format gefunden wird, kann der Server z. B. mit 406 Not Acceptable antworten.

Diese Form der Aushandlung erklärt, wie ein Client aktiv Einfluss auf die Darstellung nimmt. Für die Lehrlogik ist wichtig: Nicht der Pfad allein bestimmt die Repräsentation, sondern oft das Zusammenspiel aus Ressource, Headern und Serverentscheidung.

Eine Ressource, mehrere Repräsentationen

Beispiel: Dieselbe Produkt-Ressource /products/42

Browser-orientiert

```
GET /products/42 HTTP/1.1  
Accept: text/html
```

```
HTTP/1.1 200 OK  
Content-Type: text/html
```

API-orientiert

```
GET /products/42 HTTP/1.1  
Accept: application/json
```

```
HTTP/1.1 200 OK  
Content-Type: application/json
```

Kernidee:

Die **Ressource** bleibt dieselbe, aber ihre **Repräsentation** kann je nach Client unterschiedlich sein.

Die Trennung von Ressource und Repräsentation ist das Kernkonzept hinter dem R in REST — „Representational State Transfer“. Eine Ressource (z.B. Produkt mit ID 42) ist eine abstrakte fachliche Entität; ihre Repräsentation ist das konkrete Format, in dem ihr Zustand übertragen wird (JSON, HTML, XML, CSV). Dieselbe Ressource kann mehrere Repräsentationen haben — der Content-Type-Header identifiziert, welche ausgeliefert wurde. Caches müssen diese Unterscheidung kennen, weil sie nicht nur URL, sondern auch Repräsentation für das Caching berücksichtigen (HTTP Vary-Header).

Warum das für HTTP-Verständnis wichtig ist

- Content-Type und Accept machen Repräsentationen explizit
- Caches müssen wissen, **welche Variante** einer Ressource gespeichert wurde
- Infrastruktur kann nur korrekt arbeiten, wenn Header die Nachricht eindeutig beschreiben
- Dieselbe fachliche Ressource kann für unterschiedliche Clients unterschiedlich dargestellt werden

Header sind nicht Beiwerk, sondern Teil der HTTP-Semantik. Sie beschreiben Formate, Aushandlung, Kompression, Caching, Sicherheit und Zustand. Ohne dieses Verständnis bleiben spätere Themen wie Content Negotiation, Cache-Control oder API-Design unvollständig.

Speaker notes

Dieses Modul schafft die Brücke zwischen Grundsyntax und späteren Spezialfällen. Caching, Sessions, Sicherheit und später auch API-Design bauen darauf auf, dass Nachrichten und Repräsentationen durch Header präzise beschrieben werden. Besonders wichtig ist dabei die Unterscheidung zwischen fachlicher Ressource und uebertragener Variante: Ein Cache muss nicht nur wissen, dass `/products/42` gespeichert wurde, sondern auch ob dies die HTML-, JSON- oder komprimierte Variante war.



HTTP Kontrolle, Caching und Sicherheit

Leitfrage: Wie bleibt HTTP trotz Zustandslosigkeit effizient und steuerbar, wenn Inhalte umgeleitet, zwischengespeichert oder erneut validiert werden müssen?

Nachdem Header, Medienformate und Repräsentationen eingefuehrt wurden, lassen sich nun ihre wichtigsten Anwendungen behandeln: Caching, Redirects, Cookies und TLS. Diese Themen sind Spezialfälle derselben HTTP-Semantik.

Szenario: Der veraltete Preis

Ein Nutzer besucht die Produktseite des Funkkopfhörers. Der Preis wird von € 79,99 auf € 69,99 gesenkt. Der Nutzer lädt die Seite erneut und sieht trotzdem weiter den alten Preis.

Aufgabe: Was ist passiert? Welche HTTP-Mechanismen sind beteiligt?

Caching speichert Responses zwischen, um Last und Latenz zu reduzieren. Cache-Control: max-age=300 erlaubt dem Browser, die Ressource für 300 Sekunden ohne Rückfrage wiederzuverwenden — der Request erreicht den Server in diesem Zeitfenster gar nicht. Dadurch kann der Nutzer kurzzeitig veraltete Daten sehen. Das ist kein Browser-Fehler, sondern exakt das beabsichtigte Verhalten auf Basis der vom Server gesetzten Direktive. Der Trade-off ist architektonisch explizit: Zu hohe max-age-Werte liefern veraltete Preise oder Lagerbestände; zu niedrige Werte erhöhen die Serverlast unnötig. ETag/Last-Modified mit Conditional Requests (If-None-Match, If-Modified-Since) bieten eine feinere Alternative: Der Cache wird validiert, ohne die vollständige Ressource erneut zu übertragen.

Szenario: Der veraltete Preis

Ein Nutzer besucht die Produktseite des Funkkopfhörers. Der Preis wird von € 79,99 auf € 69,99 gesenkt. Der Nutzer lädt die Seite erneut und sieht trotzdem weiter den alten Preis.

Aufgabe: Was ist passiert? Welche HTTP-Mechanismen sind beteiligt?

Der Browser hat die vorherige Response gecacht. Der Cache-Control: max-age=300 Header sagte: "Diese Daten sind 5 Minuten gültig." Innerhalb dieser Frist fragt der Browser **gar nicht** beim Server nach.

Szenario: Der veraltete Preis

Ein Nutzer besucht die Produktseite des Funkkopfhörers. Der Preis wird von € 79,99 auf € 69,99 gesenkt. Der Nutzer lädt die Seite erneut und sieht trotzdem weiter den alten Preis.

Aufgabe: Was ist passiert? Welche HTTP-Mechanismen sind beteiligt?

Der Browser hat die vorherige Response gecacht. Der Cache-Control: max-age=300 Header sagte: "Diese Daten sind 5 Minuten gültig." Innerhalb dieser Frist fragt der Browser **gar nicht** beim Server nach.

Architekturentscheidung: Wie lange sollen Produktdaten gecacht werden? Zu lang -> veraltete Preise. Zu kurz -> jeder Seitenaufruf belastet den Server.

Caching: Ebenen und Mechanismen

Cache-Ebene	Wo?	Vorteil
Browser-Cache	Lokal auf dem Gerät	Kein Netzwerk-Request -> schnellste Variante
Proxy/CDN-Cache	Edge-Server im Netzwerk	Reduziert Last auf Origin, niedrige Latenz
Application-Cache	Im Server (Redis, Memcached)	Vermeidet wiederholte Datenbankabfragen

Jede Ebene spart Arbeit an einer anderen Stelle:

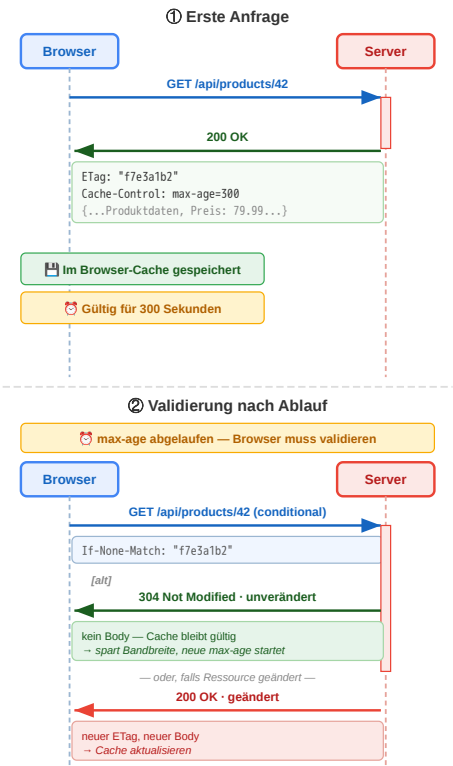
- Browser-Cache spart Netzverkehr
- CDN spart Origin-Last
- Application-Cache spart Datenbank- oder Rechenaufwand

Caches existieren auf mehreren Ebenen. Browser-Caches vermeiden Netzverkehr, CDN- oder Proxy-Caches entlasten den Ursprungsserver, serverseitige Application-Caches vermeiden teure Berechnungen oder Datenbankzugriffe. Alle drei Ebenen verbessern Performance, aber mit unterschiedlichen Trade-offs.

Cache-Steuerung über Header

Header	Funktion	Beispiel
Cache-Control: max-age=3600	1 Stunde gültig	Statische Assets
Cache-Control: no-cache	Immer validieren	Produktdaten, Preise
Cache-Control: no-store	Nie cachen	Bankdaten
Cache-Control: public	CDN darf cachen	Öffentliche Inhalte
Cache-Control: private	Nur Browser	Nutzerdaten
ETag	Ressourcen-Fingerprint	"a1b2c3d4"
Last-Modified	Letzter Änderungszeitpunkt	Mon, 23 Mar 2026

Wie der Browser entscheidet: Cache vorhanden? → max-age abgelaufen? → Validierungsrequest mit If-None-Match: ETag
Validierung: Server antwortet 304 Not Modified (kein Body, spart Bandbreite) oder 200 OK + neuer Body + neuer ETag.

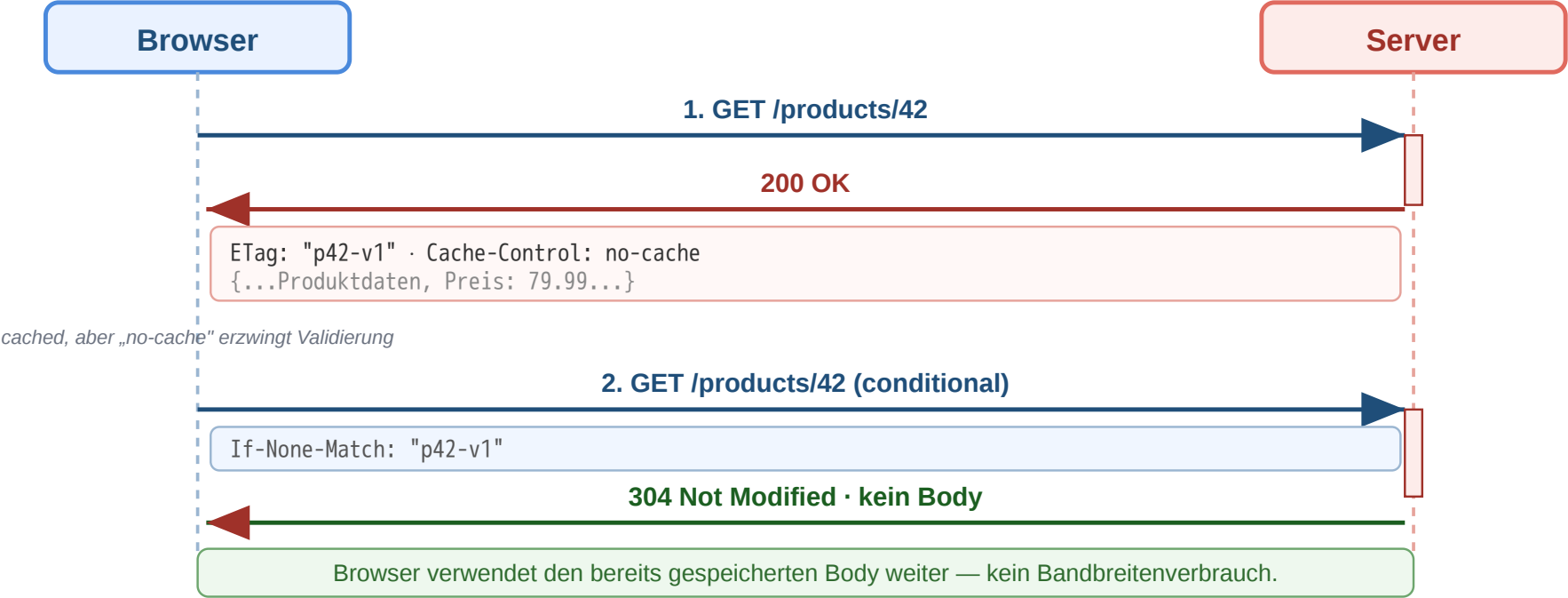


Speaker notes

Cache-Control steuert, ob und wie lange eine Response gespeichert werden darf. ETag und Last-Modified ermöglichen Validierungsrequests. Bei einer erfolgreichen Validierung kann der Server 304 Not Modified senden, sodass der Client seinen vorhandenen Body weiterverwendet.



Workflow: Conditional Request und Cache-Validierung



Schritt	Bedeutung
ETag empfangen	Client kennt Version der Repräsentation
If-None-Match senden	Client fragt: Hat sich etwas geändert?
304 Not Modified	Kein neuer Body nötig, Cache bleibt gültig

Dieser Workflow zeigt die typische Cache-Validierung für veränderliche Ressourcen. Der Client speichert eine Repräsentation mit ETag und fragt später bedingt erneut an. Wenn die Version unverändert ist, spart 304 Not Modified Bandbreite, weil kein neuer Body gesendet werden muss. MDN:

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/If-None-Match> und <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status/304>

Interpretationsaufgabe: Caching-Strategie

Aufgabe: Welchen Cache-Control-Header würden Sie für diese Ressourcen des Online-Shops setzen?

Ressource	Ihr Vorschlag?
app.v3.2.1.js (versionierte JS-Datei)	
/api/products/42 (Produktdaten mit Preis)	
/api/cart (Warenkorb des eingeloggten Nutzers)	
logo.png (Shop-Logo)	

Speaker notes

Die richtige Cache-Strategie hängt vom Datentyp ab. Versionierte statische Dateien sind nahezu ideal für lange Cache-Zeiten. Personalisierte oder sensible Daten sollten nicht im gemeinsamen Cache landen. Veränderliche Daten profitieren oft von Validierung statt unbedingter Frische.



Interpretationsaufgabe: Caching-Strategie

Aufgabe: Welchen Cache-Control-Header würden Sie für diese Ressourcen des Online-Shops setzen?

Ressource	Ihr Vorschlag?
app.v3.2.1.js (versionierte JS-Datei)	
/api/products/42 (Produktdaten mit Preis)	
/api/cart (Warenkorb des eingeloggten Nutzers)	
logo.png (Shop-Logo)	

Ressource	Header	Begründung
app.v3.2.1.js	public, max-age=31536000, immutable	Version im Dateinamen → ändert sich nie
/api/products/42	public, no-cache + ETag	Preise können sich ändern, Validierung nötig
/api/cart	private, no-store	Nutzerspezifisch, darf nicht im CDN landen
logo.png	public, max-age=86400	Ändert sich selten, 1 Tag reicht

Redirects und das Post/Redirect/Get-(PRG)-Pattern

Ziel: Nach POST /orders zeigt der Browser die Bestätigungsseite. Drückt der Nutzer **Reload**, fragt der Browser: "*Formular erneut senden?*" — und sendet bei Bestätigung den POST **noch einmal**. Ergebnis: doppelte Bestellung. Auch *Zurück* → *Vor* oder das Speichern als Lesezeichen löst dieses Verhalten aus.

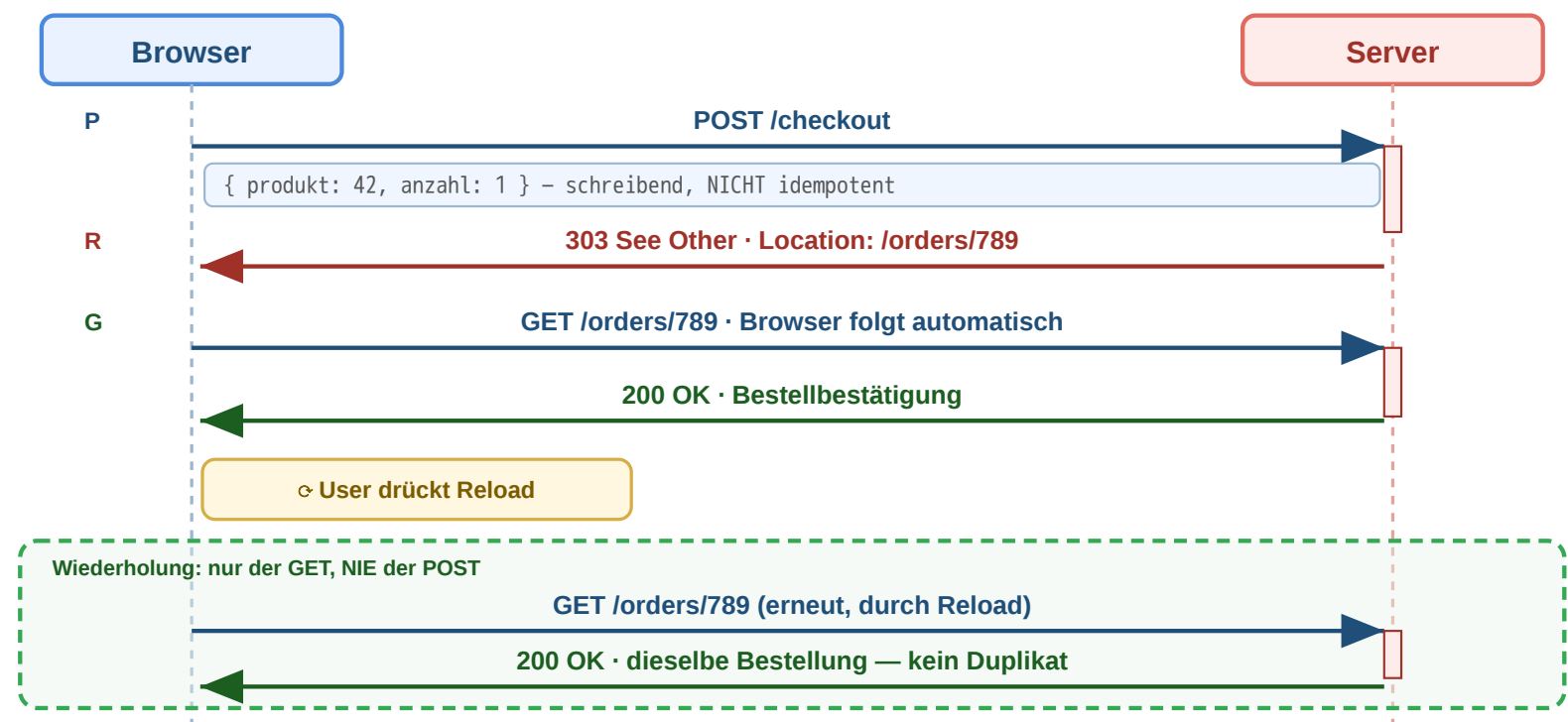
Wir wollen: den POST genau einmal ausführen und danach eine harmlose, reload-feste GET-Seite anzeigen.

Redirects verweisen Clients auf eine andere URI. 303 See Other ist besonders wichtig nach einem schreibenden POST, weil der Browser anschließend per GET auf eine bestaetigende Ressource wechselt. Dieses Muster heißt Post/Redirect/Get und verhindert doppelte Formularverarbeitung beim Aktualisieren der Seite.

Redirects und das Post/Redirect/Get-(PRG)-Pattern

Ziel: Nach POST /orders zeigt der Browser die Bestätigungsseite. Drückt der Nutzer **Reload**, fragt der Browser: "Formular erneut senden?" — und sendet bei Bestätigung den POST **noch einmal**. Ergebnis: doppelte Bestellung. Auch Zurück → Vor oder das Speichern als Lesezeichen löst dieses Verhalten aus.

Wir wollen: den POST genau einmal ausführen und danach eine harmlose, reload-feste GET-Seite anzeigen.



Code	Semantik	Typischer Einsatz
301	Permanent verschoben	Domain-Umzug
302/307	Temporär woanders	A/B-Test, Wartung
303	Nach POST auf GET	Post/Redirect/Get (PRG)
308	Permanent, Methode bleibt	API-Migration

Post/Redirect/Get (PRG) verhindert doppelte Bestellungen: Nach POST antwortet der Server mit 303 See Other → der Browser folgt per GET → Neuladen der Seite wiederholt nur den GET, nicht den POST.

Workflow: Redirects in der Praxis



Methode darf wechseln

POST → GET in der Praxis historisch üblich

Methode & Body bleiben

strikt nach RFC — wichtig für APIs

Permanent

Caches und
Crawler sollen
URL ersetzen

301 Moved Permanently

Form-URL dauerhaft umgezogen

```
→ POST /kontaktformular
{ name, email, nachricht }
← 301 Moved Permanently
Location: /v2/kontakt
→ GET /v2/kontakt ⚠ POST→GET!
← 200 OK (Body verloren)
```

→ **Browser wechselt POST → GET — Form-Daten weg!**
Bei reinem GET (z.B. HTTP → HTTPS) bleibt GET. Für POST-APIs: 308.

308 Permanent Redirect

API-Migration v1 → v2

```
→ POST /api/v1/orders
{ produkt: 42, anzahl: 1 }
← 308 Permanent Redirect
Location: /api/v2/orders
→ POST /api/v2/orders { produkt: 42 }
← 201 Created
```

→ **Methode UND Body bleiben — POST bleibt POST.**
Clients sollten neue URL persistent übernehmen.

Temporär

URL nur jetzt
anders, später
wieder Original

302 Found

POST bei abgelaufener Session

```
→ POST /api/order (Session weg)
{ produkt: 42, anzahl: 1 }
← 302 Found
Location: /login
→ GET /login ⚠ POST→GET!
← 200 OK · Login-Formular
```

→ **POST-Daten weg — Bestellung muss nach Login neu eingegeben werden!**
Für temporäre Verlegung von Schreib-Endpoints daher 307.

307 Temporary Redirect

Wartung · Failover

```
→ POST /api/orders
{ produkt: 42 }
← 307 Temporary Redirect
Location: https://b.api.example/orders
→ POST b.api.example/orders { produkt: 42 }
← 201 Created
```

→ **Methode & Body bleiben — Originaladresse merken.**
Ideal für Wartungsfenster, Failover, A/B auf POST.



Sonderfall 303 See Other

— erzwingt nach POST einen GET zur Bestätigungsseite (Post/Redirect/Get).

„Dein POST war erfolgreich, hol die Antwort hier per GET“ — macht Reload unschädlich (siehe vorige Folie).

Ziel: Eine Ressource ist nicht (mehr) unter der angefragten URL erreichbar — etwa nach Domain-Umzug, URL-Bereinigung, A/B-Test, Sprach-/Geo-Weiterleitung oder als Antwort auf einen schreibenden POST. Der Client soll **automatisch zur richtigen URL geleitet** werden, ohne dass der Nutzer eingreifen muss; Die Matrix macht zwei voneinander unabhängige Dimensionen sichtbar: **Permanenz** (cachebar, von Suchmaschinen übernommen) versus **Methodensemantik** (Suchmaschinen und Caches sollen die Verschiebung korrekt interpretieren). (darf der Client bei der Weiterleitung von POST auf GET wechseln, oder muss Methode und Body erhalten bleiben). 301 ist der Klassiker für HTTP → HTTPS und Domain-Umzüge; Browser folgen mit GET und Crawler aktualisieren ihren Index. 302 ist die typische temporäre Weiterleitung, etwa zur Login-Wand oder für Sprach-/Geo-Routing — historisch uneinheitlich, weil viele Implementierungen POST → GET wechselten, obwohl die RFC das nicht vorschreibt. 307 und 308 wurden eingeführt, weil diese Inkonsistenz für APIs nicht akzeptabel ist: Beide garantieren, dass Methode und Body unverändert weitergereicht werden — 307 temporär, 308 permanent. 303 ist der Sonderfall, der das PRG-Muster der vorigen Folie erst möglich macht: Anders als 301/302 schreibt 303 explizit vor, dass der nächste Request ein GET sein muss — der Server signalisiert damit „dein POST war erfolgreich, hol die Antwort hier per GET“. MDN: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status/303> und <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Redirections>

Lang laufende Tasks: 202 Accepted und Polling

Was tut der Server, wenn die Bearbeitung Sekunden oder Minuten dauert (CSV-Export, Video-Transcoding, LLM-Batch-Job)? Die Verbindung offen halten ist teuer und unzuverlässig. Das Standard-Muster ist ein **asynchroner Job mit Status-Polling** — eine Verkettung der bekannten Redirect-Bausteine.

```
# 1) Client startet den Task
POST /api/exports HTTP/1.1
Content-Type: application/json

{ "format": "csv", "range": "2026-Q1" }

# 2) Server quittiert SOFORT, ohne zu warten
HTTP/1.1 202 Accepted
Location: /api/jobs/abc123
Retry-After: 5

# 3) Client pollt den Job-Status (alle ~5 s)
GET /api/jobs/abc123 HTTP/1.1

HTTP/1.1 200 OK
Content-Type: application/json
{ "status": "processing", "progress": 0.42 }

# ... weitere Polls ...

# 4) Fertig → Server schickt 303 zum Ergebnis
HTTP/1.1 303 See Other
Location: /api/exports/abc123.csv

# 5) Client folgt automatisch (PRG-Mechanik)
GET /api/exports/abc123.csv HTTP/1.1

HTTP/1.1 200 OK
Content-Type: text/csv
```

Baustein	Funktion
202 Accepted	„Auftrag angenommen, Bearbeitung läuft“
Location	URL der Job-Ressource (nicht des Ergebnisses!)
Retry-After	Empfohlenes Poll-Intervall in Sekunden
Job: 200 + status	Zwischenstand: queued/processing/done/failed
Job: 303 See Other	Endzustand → Location zeigt auf das fertige Artefakt

Wann nutzt man das?

- Server-seitig teure Berechnungen (Reports, ML-Inference, Renderings)
- Asynchrone Pipelines, die ohnehin in einem Worker laufen
- LLM-Batch-APIs (Anthropic, OpenAI) — gleiche Mechanik

Verwandte Muster

- *Long Polling*: Server hält die Anfrage offen, bis sich etwas ändert
- *WebSocket / SSE*: persistente Verbindung, Server pusht Events
- *Webhook*: Client gibt Callback-URL an, Server ruft zurück

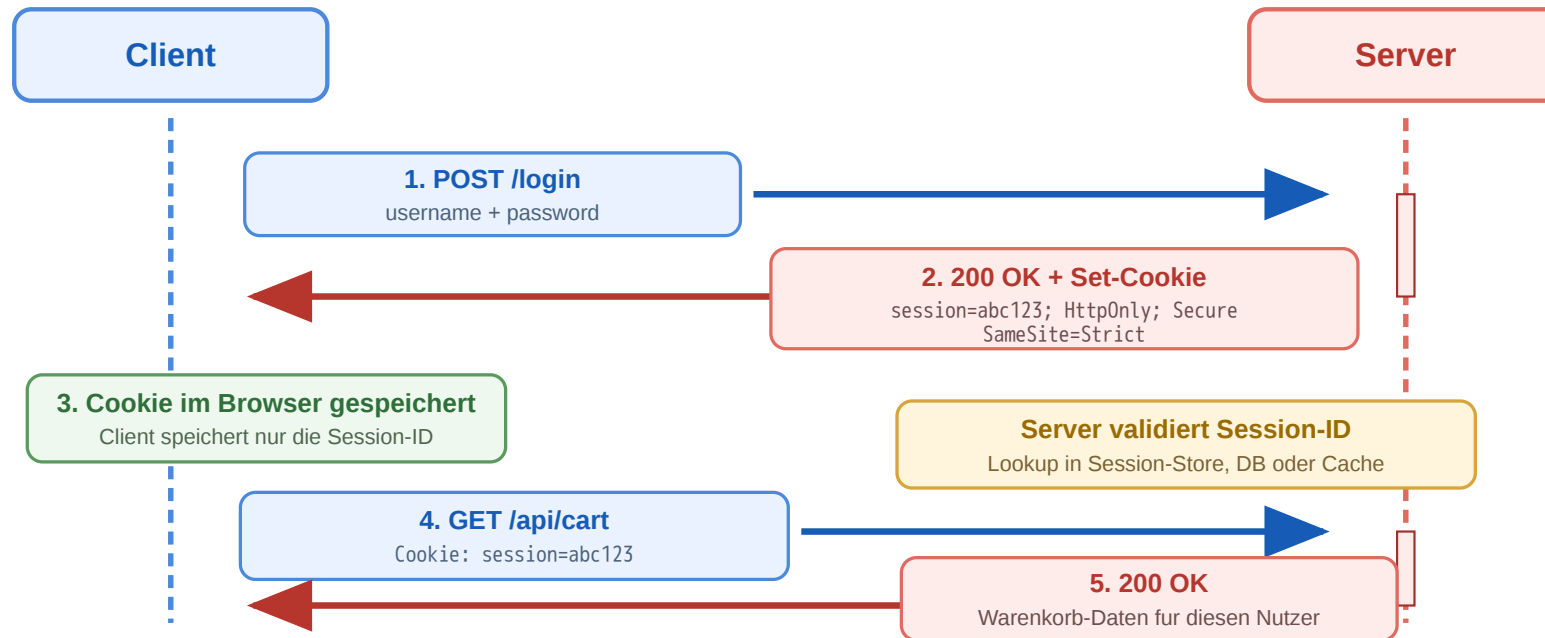


Faustregel: Sobald die Bearbeitung länger als ein Browser-Timeout (≈ 30 s) dauern kann, in Job-Ressource auslagern und per Polling beobachten.

Das Muster ist alt und bewährt — es trennt die Frage „Auftrag angekommen?“ (sofortige Antwort) von „Auftrag fertig?“ (kann lange dauern) sauber. Der entscheidende Punkt: Die Location aus dem 202 Accepted zeigt *nicht* auf die fertige Ressource, sondern auf eine *Job-Ressource*, die ihrerseits einen eigenen Lebenszyklus hat (queued → processing → done/failed). Das ist konzeptionell wichtig, weil die Job-Ressource cachebar, abbruchbar (DELETE /api/jobs/abc123) und mit eigenen Statuscodes adressierbar ist. Der 303 See Other am Ende ist dieselbe PRG-Mechanik wie bei einem normalen Form-Submit — der Client folgt mit GET zur fertigen Repräsentation. Vorsicht beim Polling-Intervall: Zu kurz erzeugt unnötige Last (jeder Poll ist ein voller Round-Trip mit Auth-Overhead), zu lang verzögert die wahrgenommene Antwortzeit. Server geben deshalb über Retry-After einen Hinweis. Echtes *Long Polling* ist eine Variante, bei der der Server die GET-Anfrage *offen hält*, bis sich der Job-Status ändert (oder ein Timeout greift); spart Round-Trips, blockiert aber Server-Verbindungen — heute meist durch SSE oder WebSockets ersetzt. **Beispiele aus der Praxis:** Anthropic's Message Batches API, OpenAI's Batch API, GitHub Actions Job-Status, AWS S3 Multipart Uploads, Stripe's asynchrone Webhooks — alle folgen einer Variante dieses Musters.

Sessions und Cookies

HTTP ist zustandslos — aber der Online-Shop braucht Login und Warenkorb. **Cookies** lösen diesen Widerspruch:



Ablauf

1. POST /login mit Credentials → Server prüft.
2. Server antwortet mit Set-Cookie:
session=abc123 und legt die Session (Warenkorb, User-ID, ...) serverseitig ab.
3. Browser speichert das Cookie für die Domain und sendet bei jedem weiteren Request automatisch Cookie: session=abc123 mit.
4. Server liest die ID, schlägt die Session nach und kennt damit Identität und Zustand des Clients. Das Cookie enthält meist nur eine **opaque ID** — der eigentliche Zustand liegt im Server (DB oder Redis).

Fundament: Same-Origin Policy — eine *Origin* ist die Kombination aus **Schema + Host + Port** (z.B. `https://shop.example:443`). Cookies gehören zu genau einer Origin:

- Der Browser sendet ein Cookie **nur zurück an die Domain, die es gesetzt hat** — nie an andere Domains.
- JavaScript auf `evil.example` kann das Cookie von `shop.example` **nicht lesen** (`document.cookie` ist origin-isoliert).
- Aber: bindet `news.example` ein Bild von `tracker.ad.net` ein, sendet der Browser dabei das Cookie von `tracker.ad.net` mit — ohne dass `news.example` es jemals sieht. Genau diese Regel ermöglicht Third-Party-Tracking (s. spätere Folien) und ist gleichzeitig die Basis, auf der die Angriffe der nächsten Folie (XSS, CSRF) ansetzen.

HTTP selbst ist zustandslos. Cookies führen dennoch wiedererkennbare Clients ein, indem der Browser kleine Wertepaar-Informationen speichert und bei späteren Requests mitsendet. Bei serverseitigen Sessions enthält das Cookie meist nur eine Session-ID; der eigentliche Zustand liegt auf dem Server. Das hat zwei Konsequenzen: Erstens ist die ID das einzige, was ein Angreifer braucht, um sich als der Nutzer auszugeben — ihr Schutz ist sicherheitskritisch. Zweitens muss der Server den Session-Store irgendwo halten; in horizontal skalierten Setups führt das zu Sticky Sessions oder einem geteilten Backend wie Redis.

Die **Same-Origin Policy** ist eine der wichtigsten Sicherheitsregeln des Browsers überhaupt. Sie isoliert Origins voneinander: Skripte einer Origin dürfen weder DOM noch Cookies einer anderen Origin lesen. Cookies sind dabei *domain-gebunden* (nicht origin-gebunden im strengen Sinne — Cookies kennen Schema/Port nicht so feingranular wie der Rest der Policy, deshalb gibt es zusätzlich das Secure-Attribut). Wichtig ist die Asymmetrie zwischen *Lesen* und *Senden*: Lesen ist strikt origin-isoliert, aber das automatische *Mitsenden* von Cookies bei eingebetteten Ressourcen oder Cross-Site-Requests passiert auch jenseits der eigenen Origin — und genau das ist die Lücke, die CSRF und Third-Party-Tracking ausnutzen. Die nachfolgenden Schutzmechanismen SameSite, HttpOnly und Secure schließen diese Lücke an unterschiedlichen Stellen.

Angriffsvektoren auf Session-Cookies — und Schutz

Das Cookie ist die Identität. Wer die Session-ID besitzt, ist für den Server der Nutzer — kein Passwort nötig. Drei Wege, an die ID zu kommen, drei Schutzmechanismen am Cookie-Attribut.

Drei Angriffe

Angriff	Wie?	Schutz-Attribut
XSS (Cross-Site Scripting)	Eingeschleustes JS liest document.cookie und exfiltriert die Session-ID an einen fremden Server.	HttpOnly — Cookie ist für JS unsichtbar
CSRF (Cross-Site Request Forgery)	Bösartige Site löst im Hintergrund einen Request an die Bank/Shop-API aus; Browser sendet Cookie automatisch mit.	SameSite=Strict / Lax — Cookie wird bei Cross-Site-Requests nicht gesendet
MITM / Sniffing	Cookie wird im Klartext über HTTP übertragen; im offenen WLAN abgreifbar.	Secure — Cookie wird nur über HTTPS gesendet

Sichere Konfiguration

Konfiguration	XSS	CSRF	Sniffing
session=abc123	✗	✗	✗
…; HttpOnly	✓	✗	✗
…; HttpOnly; Secure	✓	✗	✓
…; HttpOnly; Secure; SameSite=Strict	✓	✓	✓

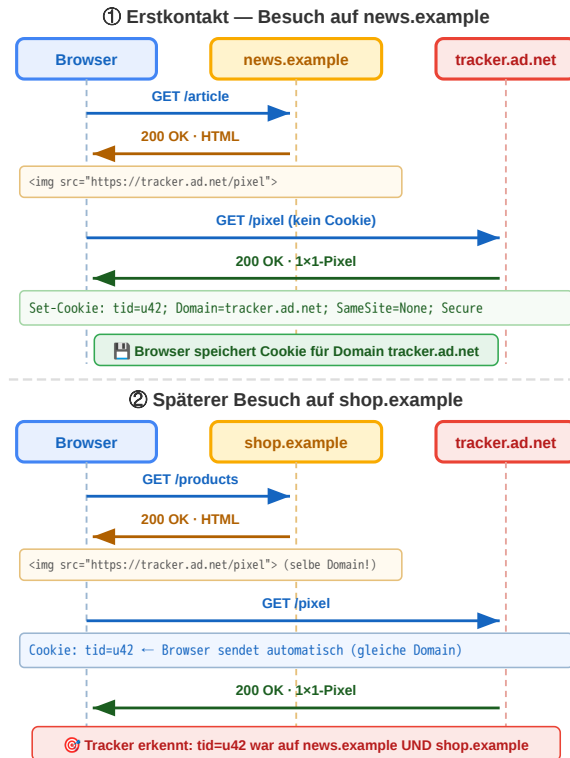
```
Set-Cookie: session=abc123;  
HttpOnly; Secure; SameSite=Strict;  
Path=/; Max-Age=3600
```

Zusätzlich: Session-ID nach Login **rotieren** (gegen Session-Fixation) · Session serverseitig **invalidieren** bei Logout · kurze **Max-Age** + Sliding-Renewal.



Session-Cookies sind ein klassisches Beispiel, bei dem Sicherheit nicht im Anwendungscode entsteht, sondern in der korrekten Konfiguration eines Protokoll-Mechanismus. Die drei Angriffsvektoren adressieren drei unterschiedliche Bedrohungsmodelle: **XSS** unterstellt, dass ein Angreifer Code im Browser des Opfers ausführt — z.B. über einen unsicheren Forum-Beitrag oder ein unsicheres Eingabefeld; `HttpOnly` macht das Cookie für JavaScript unsichtbar und schließt den direkten Diebstahl aus. **CSRF** unterstellt, dass der Nutzer auf einer fremden Site landet, die einen Request an die geschützte Anwendung auslöst — der Browser sendet das Session-Cookie automatisch mit, weil es zur Zieldomain gehört; `SameSite=Lax` oder `Strict` verhindert genau das. **Sniffing** unterstellt einen passiven Angreifer im Netzwerk; `Secure` zwingt den Browser, das Cookie nur über TLS zu senden. Wichtig ist, dass diese drei Attribute orthogonal sind — jedes blockiert genau einen Vektor, und nur die Kombination aller drei plus serverseitige Hygiene (ID-Rotation nach Login, Logout-Invalidierung, kurze Laufzeiten) ergibt eine belastbare Session-Implementierung. Wer Single-Page-Apps mit Cross-Site-API-Calls baut, muss zusätzlich CSRF-Token oder ein Double-Submit-Pattern einsetzen, weil `SameSite=Strict` dort zu restriktiv wäre.

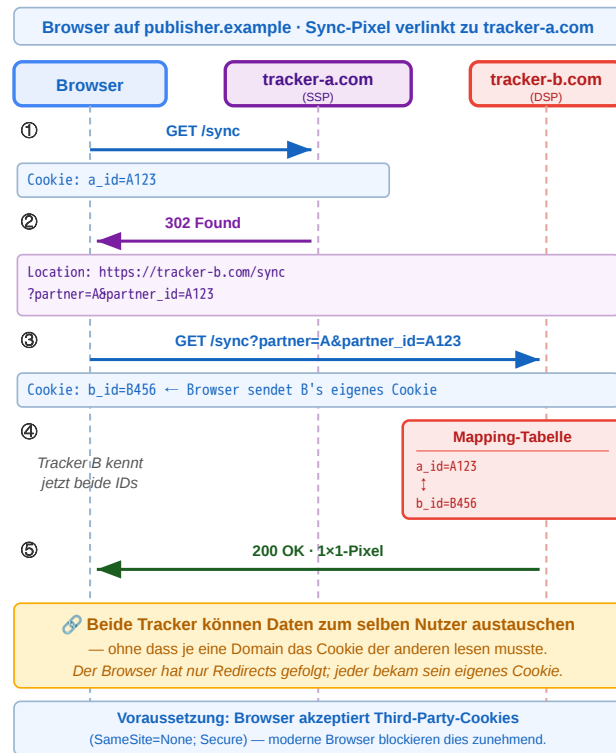
Cookie-Tracking I — Third-Party-Cookies



1. Mehrere Websites binden eine Ressource derselben Tracking-Domain ein (Bild, Skript, Werbe-Pixel, Social-Plugin).
 2. Beim ersten Kontakt setzt der Tracker ein Cookie für **seine eigene Domain** — z.B. `tracker.ad.net`.
 3. Bei jedem weiteren Besuch einer Site, die dieselbe Tracker-Ressource einbindet, sendet der Browser das Cookie automatisch mit.
 4. Der Tracker erkennt: derselbe Browser war auf `news.example` **und** `shop.example`.
- Same-Origin bleibt gewahrt: Jede Domain liest nur ihre eigenen Cookies. Tracking funktioniert dadurch, dass viele Sites **dieselbe** Tracker-Domain einbinden — nicht weil Cookies geteilt werden.

Third-Party-Cookies sind die einfachste Form von Cross-Site-Tracking. Der Trick ist nicht, dass Cookies geteilt werden — der Browser sendet Cookies nur an genau die Domain, die sie gesetzt hat. Der Trick ist, dass viele unabhängige Websites Ressourcen von derselben Tracker-Domain einbinden (Werbenetz, Analytics, Social-Buttons). Aus Sicht des Trackers entsteht so ein zusammenhängendes Profil über Sites hinweg, obwohl die Sites selbst nichts voneinander wissen. Moderne Browser (Safari ITP, Firefox ETP, Chrome Privacy Sandbox) beschränken Third-Party-Cookies inzwischen aggressiv — wer trotzdem trackt, weicht auf Mechanismen wie Cookie-Syncing oder First-Party-Bounce-Tracking aus.

Cookie-Tracking II — ID-Sync über Redirects



Problem: Tracker A und B haben jeweils ihre eigene ID für denselben Nutzer ($a_id=A123$, $b_id=B456$) — kennen aber die ID des anderen nicht. Same-Origin verbietet das direkte Lesen.

Lösung — Cookie-Syncing:

1. Sync-Pixel von Tracker A wird geladen; A liest sein Cookie.
2. A antwortet mit 302 Redirect zu Tracker B — A's eigene ID steckt **im URL-Parameter** ($?partner_id=A123$).
3. Browser folgt zum Redirect-Ziel B und sendet dabei **B's eigenes Cookie** mit; in der URL liegt A's ID.
4. B speichert die Zuordnung $A123 \leftrightarrow B456$ in seiner Mapping-Tabelle.

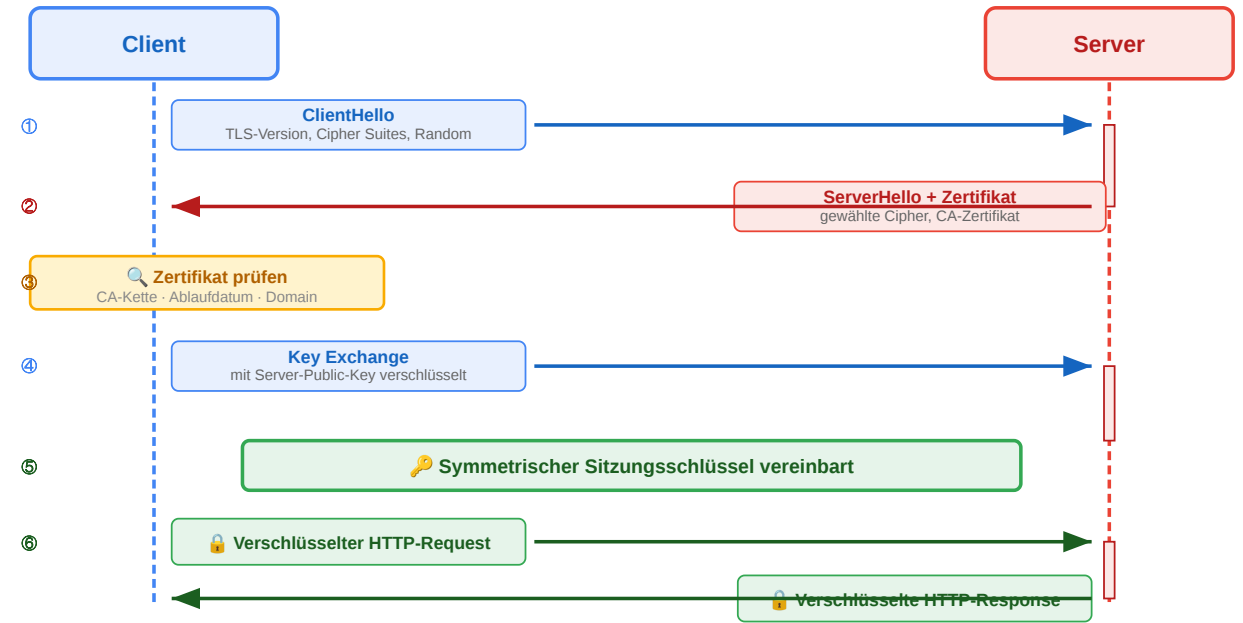
Beide Tracker können nun Daten zum selben Nutzer austauschen, **ohne dass je eine Domain das Cookie der anderen las**. Voraussetzung: der Browser akzeptiert Third-Party-Cookies — was moderne Browser zunehmend blockieren.

Cookie-Syncing (auch *Cookie-Matching* oder *ID-Matching*) ist der Mechanismus hinter Real-Time-Bidding und Werbenetzen. Er löst genau das Problem, dass eine Site die Cookie-ID einer Tracker-Domain nicht kennt: Statt das Cookie direkt zu teilen, wird die ID als URL-Parameter über eine Redirect-Kette zur Partner-Domain weitergereicht. Der Browser folgt jedem Redirect automatisch und sendet dabei zur jeweiligen Zieldomain deren eigenes Cookie. So bekommt der Empfänger zwei Informationen gleichzeitig: die fremde ID aus dem URL-Parameter und seine eigene aus dem mitgeschickten Cookie — und kann beide in einer Mapping-Tabelle verknüpfen. Same-Origin wird formal eingehalten; die Identitätsverknüpfung ist aber faktisch erfolgt. Dieser Mechanismus erklärt auch, warum Privacy-Maßnahmen wie Third-Party-Cookie-Blockaden, SameSite-Defaults und Bounce-Tracking-Erkennung in den letzten Jahren verschärft wurden.

HTTPS und TLS

Eigenschaft	HTTP	HTTPS
Vertraulichkeit	Klartext lesbar	Verschlüsselt
Integrität	Manipulierbar	Manipulation erkannt
Authentizität	Keine Garantie	Zertifikat bestätigt

Diskussionsfrage: Ein Angreifer im selben WLAN liest den Netzwerkverkehr mit. Welche der drei TLS-Eigenschaften schützt hier — und welche nicht?
(Hinweis: TLS schützt den Transport, nicht den Endpunkt.)



HTTPS bedeutet HTTP über TLS (Transport Layer Security). TLS schützt Vertraulichkeit durch Verschlüsselung, Integrität durch Manipulationsschutz und Authentizität durch Zertifikate. Es schützt die Transportstrecke, aber nicht automatisch den Browser selbst oder unsichere Anwendungslogik.

Zusammenfassung — HTTP in der Praxis

Semantik statt Syntax

- **Methoden** kommunizieren Absicht — GET liest, POST schreibt nicht-idempotent, PUT/DELETE sind idempotent.
- **Statuscodes** sind die maschinenlesbare Antwort — 2xx Erfolg, 3xx Weiterleitung, 4xx Client-Fehler, 5xx Server-Fehler.
- **Header** steuern Format, Aushandlung, Caching, Auth und Sessions — kein Beiwerk.

Performance: Caching mit Trade-off

- Cache-Control: max-age reduziert Last und Latenz — Preis: zeitweise veraltete Daten.
- ETag + If-None-Match ermöglichen Validierung ohne erneute Übertragung (304 Not Modified).
- Cache-Ebenen wirken kumulativ: Browser, CDN/Proxy, Application-Cache.

Navigation und Schreiboperationen

- **PRG** (POST → 303 → GET) verhindert doppelte Verarbeitung bei Reload.
- **301/308** dauerhaft, **302/307** temporär; **307/308** erhalten Methode und Body — Pflicht für APIs.

Zustand trotz Zustandslosigkeit

- **Cookies** stabilisieren Sessions und werden domain-gebunden mitgesendet (Same-Origin).
- Sicherheits-Attribute orthogonal: HttpOnly (XSS), SameSite (CSRF), Secure (Sniffing).
- Dieselbe Mechanik ermöglicht Tracking — direkt über Third-Party-Cookies oder per ID-Sync via Redirect.

Transport: HTTPS / TLS

- Vertraulichkeit, Integrität, Authentizität — aber nur für die Leitung, nicht für die Anwendungslogik.

HTTP bleibt simpel auf Protokollebene, wird aber durch Header und Statuscodes zum ausdrucksstarken Steuerungsmechanismus. Caching ist der wichtigste Performance-Hebel — aber jede Caching-Entscheidung ist ein Trade-off zwischen Aktualität und Last. Redirects steuern Navigation, PRG verhindert Doppelverarbeitung. Cookies ermöglichen Zustand trotz Zustandslosigkeit — aber nur mit korrekter Konfiguration. TLS sichert den Transport.

Diese Folie fasst das HTTP-Kapitel zusammen, bevor wir HTTP-Semantik gezielt auf API-Design anwenden und anschließend REST als systematisierende Architekturidee betrachten. Die wichtigste Botschaft: HTTP ist nicht nur Transport, sondern ein semantisches Protokoll. Methoden, Statuscodes und Header bilden gemeinsam ein Vokabular, das Infrastruktur (Browser, Caches, CDNs, Load Balancer, Monitoring) ohne Anwendungswissen verstehen kann. Wer dieses Vokabular korrekt verwendet, gewinnt Cachebarkeit, Wiederholbarkeit, Skalierbarkeit, Sicherheit und Beobachtbarkeit fast geschenkt; wer es ignoriert (z.B. immer 200 OK mit Fehler-JSON, alles über POST), verliert genau diese Vorteile. Die nächsten Kapitel zeigen erst, wie diese Bausteine in API-Designs konkret eingesetzt werden, und dann, wie REST sie zu Architekturprinzipien systematisiert.

HTTP-Semantik für APIs

Leitfrage: Wie helfen Methoden, Statuscodes und Header dabei, API-Verhalten ohne Zusatzprotokolle klar und maschinenlesbar zu machen?

Speaker notes

Eine gute API nutzt HTTP-Semantik vollständig aus. Methoden, Statuscodes und Header bilden zusammen den Vertrag zwischen Client und Server. Je klarer dieser Vertrag ist, desto weniger Sonderwissen braucht ein Client.



Was ist an dieser API-Antwort falsch?

Ein Client sendet eine Bestellung an den Online-Shop:

```
POST /api/orders HTTP/1.1
Content-Type: application/json

{"product_id": 42, "quantity": 2}
```

Der Server antwortet:

```
HTTP/1.1 200 OK
Content-Type: application/json

{"id": 789, "status": "created"}
```

Aufgabe: Die Bestellung wurde erfolgreich erstellt. Trotzdem hat die Response mindestens drei Probleme. Welche?

Problem	Warum	Besser
200 OK statt 201 Created	200 sagt „gelesen“, nicht „erzeugt“	201 Created
Kein Location-Header	Client weiß nicht, wo die neue Ressource liegt	Location: /api/orders/789
Kein Cache-Control	Bestellungen dürfen nicht gecacht werden	Cache-Control: no-store

~~K~~orrigierte Version

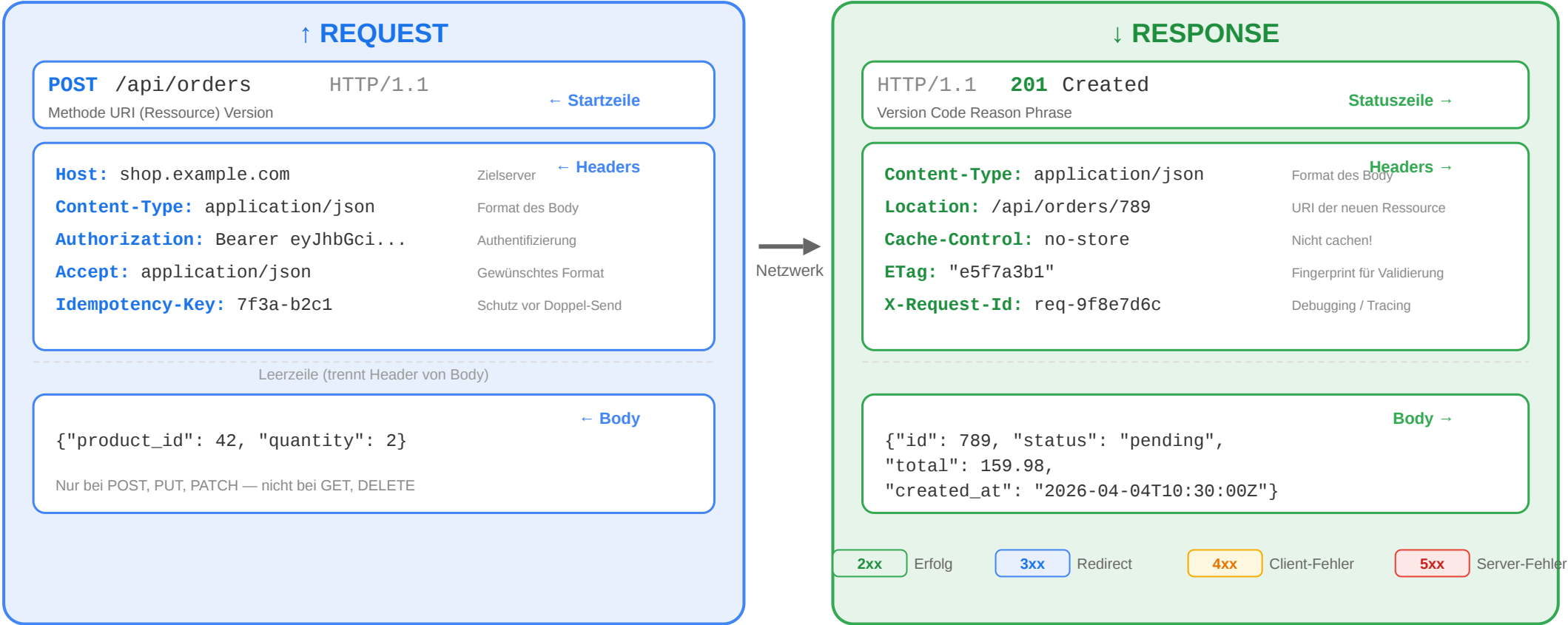
HTTP/1.1 201 Created
Location: /api/orders/789
Content-Type: application/json
Cache-Control: no-store

```
{"id": 789, "status": "pending", "total": 159.98}
```

Beim erfolgreichen Erzeugen einer Ressource ist 201 Created der passende Statuscode. Ein Location-Header verweist auf die neue Ressource. Zusätzlich sollte klar sein, ob eine solche Response gespeichert werden darf; für Bestellungen ist no-store oft sinnvoll.

Der vollständige Request-Response-Zyklus

Anatomie: HTTP Request und Response



Der Zyklus umfasst Startzeilen, Header und optionale Bodies auf beiden Seiten. Requests enthalten typischerweise Ziel, Semantik und Metadaten; Responses enthalten Ergebnis, Beschreibungsdaten und die eigentliche Repräsentation.

Statuscodes richtig einsetzen

Situation	Richtiger Code	Häufiger Fehler
Ressource erfolgreich gelesen	200 OK	—
Ressource erfolgreich angelegt	201 Created + Location	200 ohne Location
Erfolgreich, aber kein Body	204 No Content	200 mit leerem Body
Ungültige Eingabedaten	400 Bad Request	500 (ist kein Server-Fehler!)
Nicht authentifiziert	401 Unauthorized	403
Authentifiziert, aber nicht berechtigt	403 Forbidden	401
Ressource nicht gefunden	404 Not Found	200 mit Fehlermeldung im Body

Faustregel:

Der Statuscode sagt dem Client, **was passiert ist**. Der Body erklärt **warum** und **was zu tun ist**.



Situation	Richtiger Code	Häufiger Fehler
Konflikt (z.B. doppelter Username)	409 Conflict	400

Die Semantik der 4xx-Codes ist präziser als oft angenommen: 401 Unauthorized bedeutet, dass die Identität fehlt oder ungültig ist — der Client soll sich neu authentifizieren. 403 Forbidden bedeutet, dass die Identität bekannt, aber die Berechtigung nicht vorhanden ist — erneutes Einloggen hilft nicht. 400 Bad Request beschreibt syntaktisch oder strukturell ungültige Eingaben, 422 Unprocessable Entity fachlich ungültige (aber korrekt formatierte) Eingaben, 409 Conflict fachliche Konflikte (z.B. Duplikat). 204 No Content signalisiert Erfolg ohne Rückgabe-Body — für DELETE und PUT gebräuchlich. Der Statuscode ist der maschinenlesbare Kern der Antwort: Caches, Load Balancer und Monitoring-Systeme reagieren auf Codes, nicht auf Body-Inhalte. Falsch gewählte Codes (z.B. 200 mit Fehlerbeschreibung im Body) machen die API für automatisierte Verarbeitung unbrauchbar.

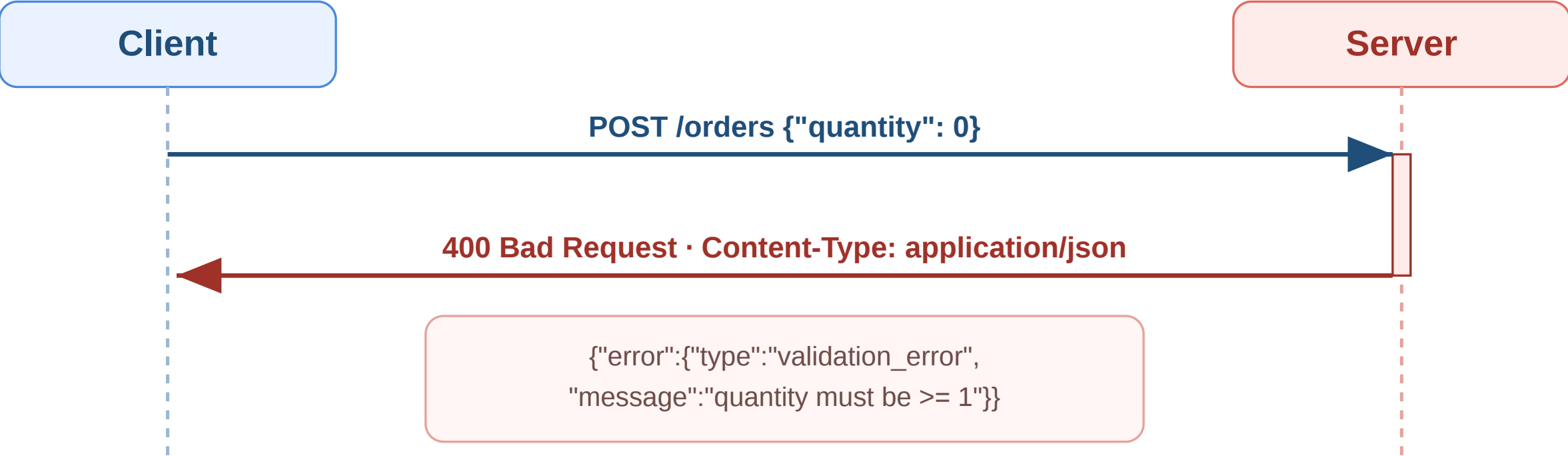
Fehlermodelle: Konsistenz ist alles

```
{
  "error": {
    "type": "validation_error",
    "message": "Die Eingabedaten sind ungültig.",
    "details": [
      { "field": "email", "code": "invalid_format",
        "message": "Keine gültige E-Mail-Adresse" },
      { "field": "quantity", "code": "out_of_range",
        "message": "Muss zwischen 1 und 100 liegen" }
    ]
  }
}
```

Prinzip	Umsetzung
Konsistente Struktur	Immer dasselbe JSON-Format für Fehler
Maschinenlesbar	type und code für automatische Verarbeitung
Menschenlesbar	message für Entwickler und Endnutzer
Spezifisch	details pro Feld bei Validierungsfehlern
Kein Stacktrace	Interne Details nie an den Client

Ein konsistentes Fehlerformat macht APIs leichter nutzbar. Maschinenlesbare Felder wie `type` oder `code` helfen bei automatischer Verarbeitung; menschenlesbare Nachrichten helfen beim Debugging. Interne Details wie `Stacktraces` gehören nicht in externe Fehlermeldungen.

Workflow: Fehler sauber kommunizieren



Situation	Typischer Code
Ungültige Eingabe	400 Bad Request oder 422 Unprocessable Content
Nicht gefunden	404 Not Found
Konflikt	409 Conflict



Situation	Typischer Code
Interner Fehler	500 Internal Server Error

Ein Fehler-Workflow besteht nicht nur aus einem Statuscode, sondern aus Statuscode plus konsistentem Fehler-Body. Der Code liefert die maschinenlesbare Klasse des Problems, der JSON-Body die Details. So können Clients sowohl automatisiert reagieren als auch sinnvolle Fehlermeldungen anzeigen. MDN: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status/400> und <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status/409>

Authentifizierung: Wer darf was?

Mechanismus	Funktionsweise	Typischer Einsatz
API Key	Statischer Schlüssel im Header	Einfache M2M-Kommunikation
Bearer Token (JWT)	Signierter Token im Authorization-Header	SPAs, Mobile Apps
OAuth 2.0	Delegierte Autorisierung über Authorization Server	„Login mit Google“
Session Cookie	Cookie mit Session-ID	Klassische server-gerenderte Apps

JWT (JSON Web Token)

```
Header.Payload.Signature  
eyJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkIjo0Mn0.abc123signature
```

Teil	Inhalt
Header	Algorithmus und Token-Typ
Payload	Claims: user_id, Rollen, Ablaufzeit
Signature	Kryptografische Signatur → Manipulation erkennbar

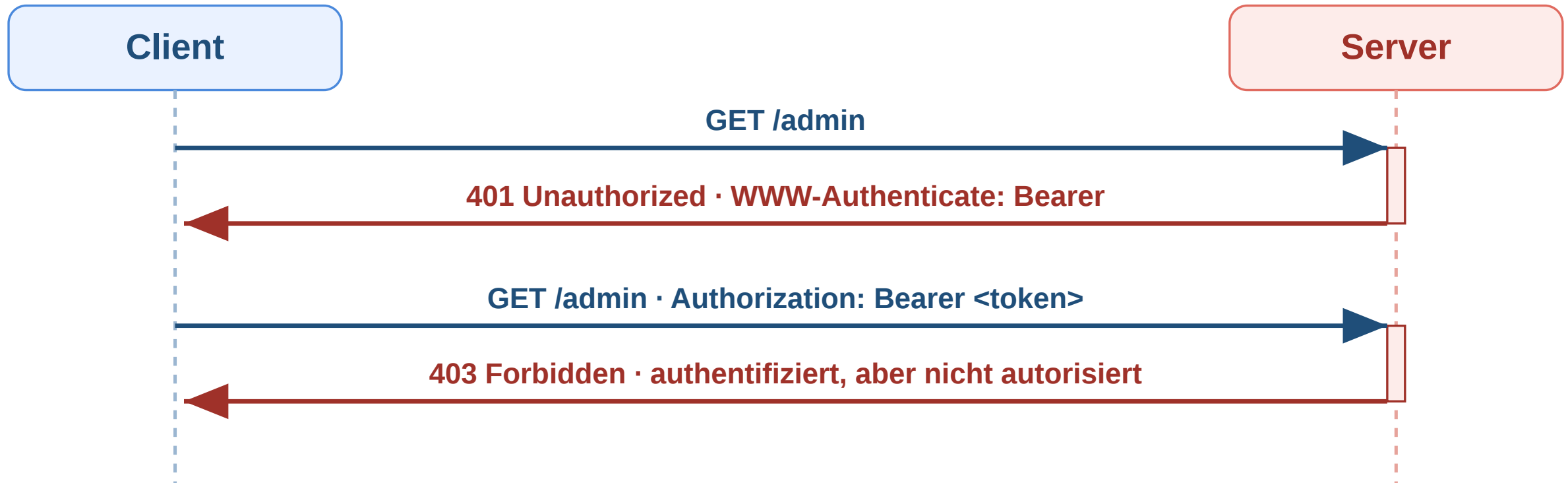
Diskussionsfrage: JWT ist stateless — der Server muss keine Session speichern. Aber: Wie widerruft man einen JWT, bevor er abläuft?

Speaker notes

Authentifizierung beantwortet die Frage, wer ein Client ist; Autorisierung beantwortet, was dieser Client tun darf. API Keys, Session Cookies, Bearer Tokens und OAuth 2.0 adressieren unterschiedliche Einsatzszenarien und Sicherheitsanforderungen. OAuth steht für Open Authorization und beschreibt ein Verfahren für delegierte Zugriffsfreigaben.



Workflow: Authentifizierung mit 401 und 403



Status	Bedeutung
401 Unauthorized	Keine gültigen Anmeldedaten vorhanden
403 Forbidden	Identität bekannt, aber Zugriff nicht erlaubt

Der Unterschied zwischen 401 und 403 ist didaktisch zentral. 401 Unauthorized fordert zunächst gültige Authentifizierung an und geht oft mit WWW-Authenticate einher. 403 Forbidden bedeutet dagegen, dass die Identität akzeptiert wurde, aber für diese Operation keine Berechtigung besteht. MDN:

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status/401> und <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status/403>

Rate Limiting und Idempotenz



Idempotenz

Methode	Idempotent?	Warum
GET	ja	Liest nur
PUT	ja	Ergebnis ist immer derselbe Zustand
DELETE	ja	Zweites Löschen hat keinen Effekt
POST	nein	Jeder Aufruf kann neue Ressource erzeugen

Szenario: Ein Nutzer klickt "Jetzt bestellen", die Verbindung bricht ab. Der Client weiß nicht, ob der Server die Bestellung schon verarbeitet hat.

Lösung: Idempotency-Key - der Server erkennt den Retry und liefert das Ergebnis des ersten Requests zurück.

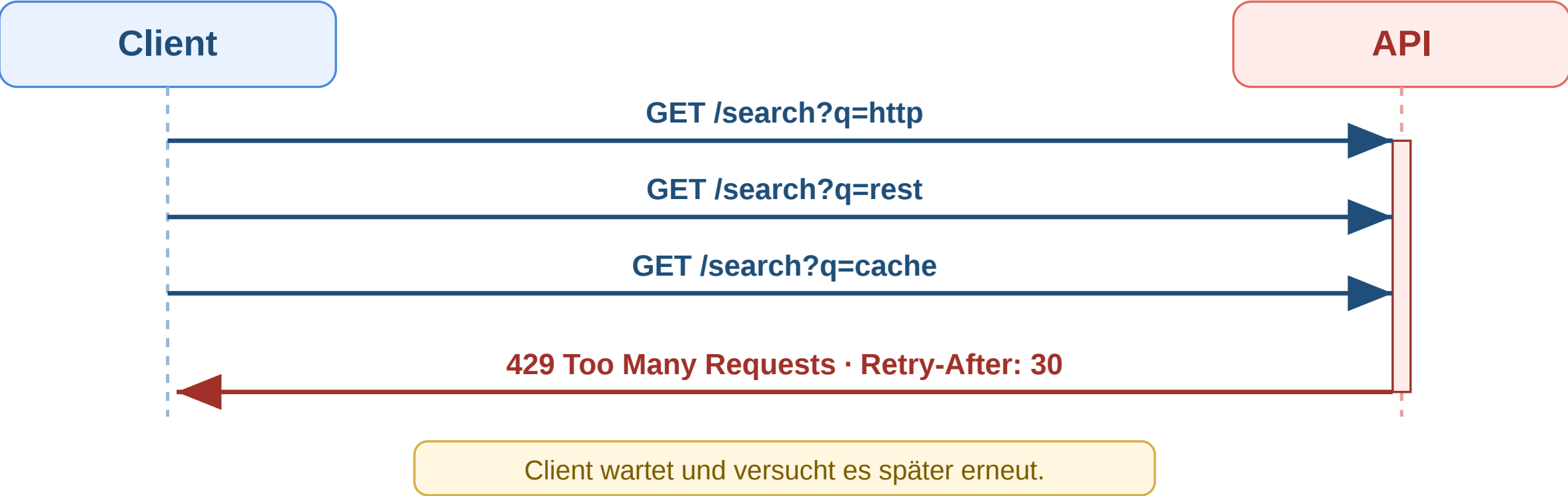
Rate Limiting

```
HTTP/1.1 429 Too Many Requests
Retry-After: 30
X-RateLimit-Limit: 1000
X-RateLimit-Remaining: 0
```

APIs begrenzen Requests pro Zeitfenster. Die Header sagen dem Client **wann** er es erneut versuchen darf.

HTTP-Semantik ist der Vertrag zwischen Client und Server. Statuscodes kommunizieren Ergebnisse, Header steuern Rate Limiting begrenzt die Anzahl von Requests innerhalb eines Zeitfensters und schützt Systeme vor Überlastung oder Missbrauch. Idempotency Keys lösen ein Caching und Authentifizierung, konsistente Fenstermodelle machen APIs nutzbar. Idempotenz-Mechanismen schützen gegen doppelte Verarbeitung. Je klarer die HTTP-Semantik genutzt wird, desto weniger Sonderlogik brauchen Clients.

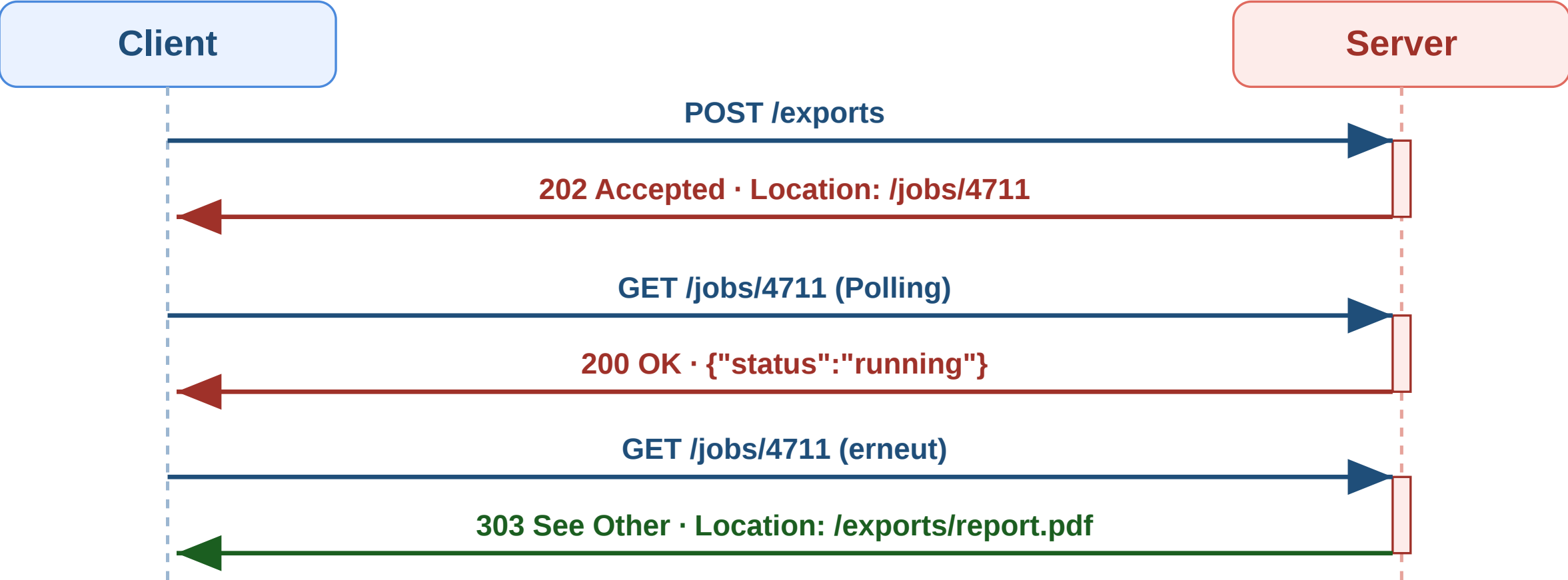
Workflow: Rate Limit und Retry-After



Header / Code	Bedeutung
429 Too Many Requests	Limit für dieses Zeitfenster erreicht
Retry-After: 30	Neuer Versuch erst nach 30 Sekunden
X-RateLimit-*	Optionale Zusatzinfos zu Limit und Restbudget

Beim Rate-Limit-Workflow kommuniziert der Server nicht nur Ablehnung, sondern auch Wiederanlaufbedingungen. Gute Clients respektieren Retry-After, reduzieren ihre Frequenz und kombinieren dies oft mit Backoff-Strategien. MDN: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status/429>

Workflow: Asynchrone Verarbeitung mit 202 Accepted



Phase	Bedeutung
202 Accepted	Auftrag angenommen, aber noch nicht abgeschlossen
Job-Ressource	Fortschritt oder Status abfragbar



Phase	Bedeutung
303 See Other	Ergebnis liegt nun unter eigener Ressource

Der asynchrone Workflow ist für längere Prozesse wichtig, etwa Exporte, Videoverarbeitung oder Batch-Jobs. 202 Accepted ist nicht gleich Erfolg des eigentlichen Jobs, sondern nur Annahme zur Verarbeitung. Eine Status-Ressource entkoppelt Start und späteres Ergebnis sauber. MDN: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status/202> und <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status/303>

REST als Architekturidee

Leitfrage: Was unterscheidet eine RESTful API von einem beliebigen HTTP-Endpunkt, der nur JSON ausliefert?

REST ist kein Protokoll, sondern ein Architekturstil für verteilte Systeme. Er baut auf Web-Grundprinzipien auf und nutzt HTTP, URIs (Uniform Resource Identifiers), Repräsentationen und Caching bewusst als Infrastrukturmechanismen.

REST kurz eingeführt

REST steht für **Representational State Transfer**.

REST entstand nicht als Framework oder Produkt, sondern als Beschreibung dafuer, **wie das Web bereits erfolgreich funktioniert**:

- Ressourcen haben stabile Adressen (/products/42)
- Clients navigieren über standardisierte HTTP-Operationen
- Server liefern **Repräsentationen** dieser Ressourcen
- Infrastruktur wie Browser, Caches und Proxies kann mitarbeiten

Entstehungskontext

- Ende der 1990er Jahre im Umfeld der Web-Architektur
- Geprägt durch **Roy Fielding**
- Ausgangsfrage: Warum skaliert das Web trotz seiner Einfachheit so gut?

Für die Praxis: REST ist zuerst ein **Programmier- und Entwurfsparadigma für verteilte Client-Server-Systeme**. Die formale Einordnung als Architekturstil kommt im naechsten Schritt.

REST (Representational State Transfer) wurde 2000 von Roy Fielding in seiner Dissertation „Architectural Styles and the Design of Network-based Software Architectures“ beschrieben — nicht als neue Erfindung, sondern als Formalisierung der Prinzipien, die das Web von Anfang an skalierbar gemacht haben. Fielding war maßgeblich an der Spezifikation von HTTP/1.1 beteiligt und analysierte im Rückblick, welche Architekturentscheidungen den Erfolg des Webs erklären. Der Name betont den Kern: Übertragen wird nicht die Ressource selbst (die ist ein abstraktes Konzept auf dem Server), sondern eine Repräsentation ihres aktuellen Zustands — identifiziert über eine URI. REST ist damit primär ein Beschreibungsmodell für HTTP-Nutzung, kein eigenständiges Protokoll.

REST: Ein Architekturstil, kein Protokoll

REST ist kein einzelnes Protokoll, sondern ein Satz von Constraints für verteilte Systeme.

Constraint	Bedeutung	Konsequenz
Client-Server	Klare Rollentrennung	Client und Server entwickeln sich unabhängig
Stateless	Jeder Request enthält alle nötigen Informationen	Server skaliert horizontal, kein Session-Zustand
Cacheable	Responses deklarieren ihre Cachebarkeit	Infrastruktur kann Caching transparent umsetzen

Constraint	Bedeutung	Konsequenz
Uniform Interface	Einheitliche Schnittstelle für alle Ressourcen	Generische Werkzeuge (Browser, curl, Proxies) funktionieren
Layered System	Client weiß nicht, ob er direkt mit dem Server spricht	Load Balancer, CDN, API Gateway transparent einfügbar
Code on Demand (optional)	Server kann ausführbaren Code liefern	JavaScript im Browser

Kernidee

REST nutzt die **vorhandene Web-Infrastruktur** (HTTP, URIs, Caching, Proxies) als Anwendungsplattform — statt ein eigenes RPC-Protokoll darüber zu bauen.

Die zentralen REST-Constraints sind Client-Server, Statelessness, Cacheability, Uniform Interface und Layered System; Code on Demand ist optional. Diese Constraints sollen Skalierbarkeit, Entkopplung und Wiederverwendbarkeit von Infrastruktur verbessern.

Welchen Constraint verletzt dieses Design?

Szenario 1: Der Online-Shop speichert den Warenkorb in einer serverseitigen Session. Beim nächsten Request muss der Client denselben Server treffen (Sticky Sessions).

Szenario 2: Die API liefert bei GET `/api/products` keinen Cache-Control-Header. Jeder Request geht bis zum Origin-Server.

Szenario 3: Der Client kennt die interne Datenbankstruktur und baut SQL-artige Queries: GET `/api/query?table=products&where=price>50`

Szenario	Verletzter Constraint	Konsequenz
1: Sticky Sessions	Stateless	Horizontale Skalierung eingeschränkt, Server-Ausfall = Session verloren
2: Kein Cache-Control	Cacheable	CDN nutzlos, unnötige Serverlast, höhere Latenz
3: SQL-artige Queries	Uniform Interface + Layered System	Client an Interna gekoppelt, keine Abstraktion möglich

Sticky Sessions verletzen das Ideal der Statelessness, fehlende Cache-Deklaration verhindert Cacheability und ein direkter Bezug auf interne Datenstrukturen verletzt das Uniform Interface. REST bewertet APIs also nicht nur nach URI-Form, sondern nach ihrem Verhalten als verteiltes System.

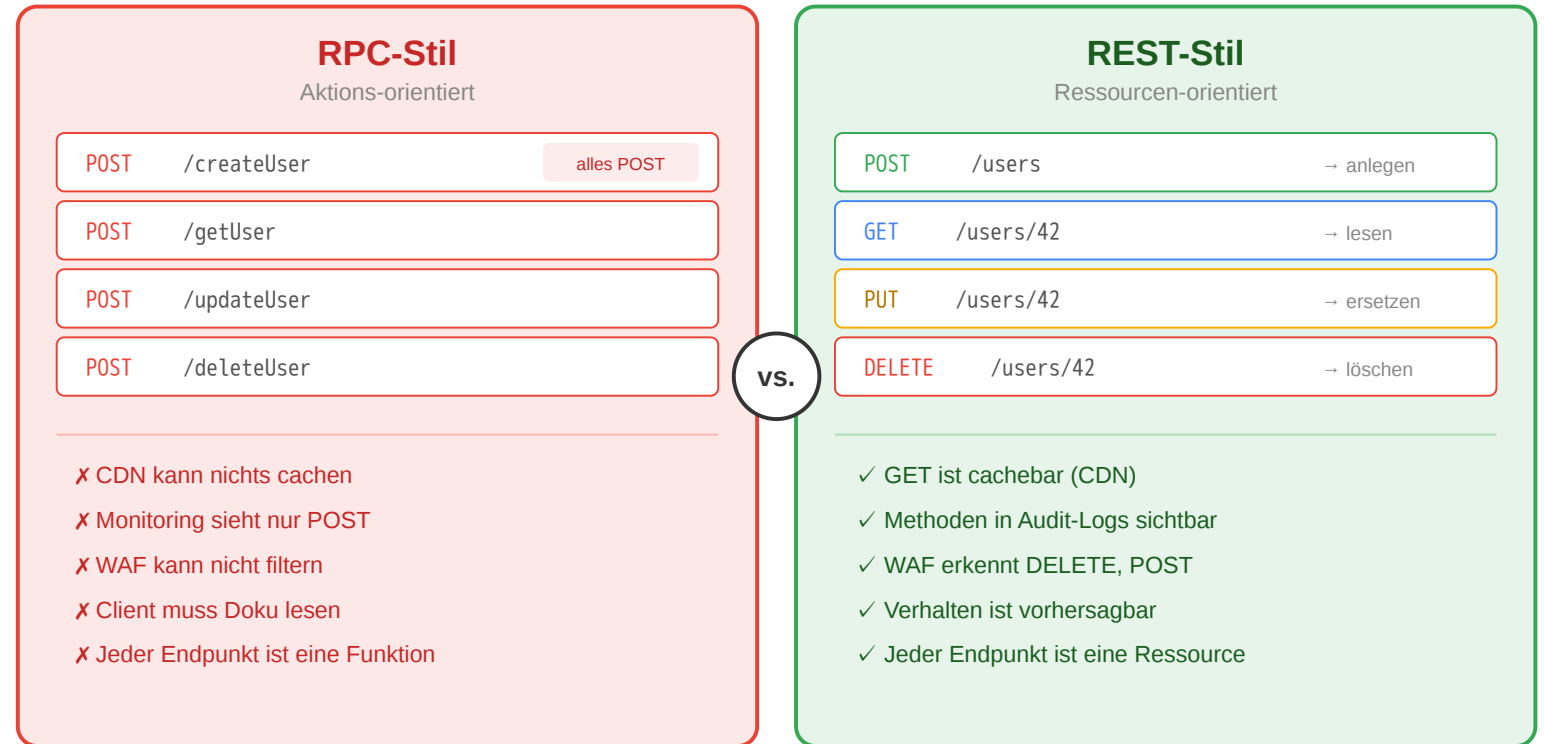
REST vs. RPC im Vergleich

RPC-Stil (aktionsorientiert)

- Jeder Endpunkt ist eine **Funktion**
- HTTP-Methode immer POST
- Infrastruktur kann nichts cachen
- Client muss Doku lesen

REST-Stil (ressourcenorientiert)

- Jeder Endpunkt ist eine **Ressource**
- HTTP-Methoden drücken Absicht aus
- GET-Requests sind cachebar
- Verhalten ist vorhersagbar



Warum das für Infrastruktur relevant ist

CDN, WAF und Monitoring verstehen HTTP-Semantik: GET wird gecacht, POST als schreibend erkannt, DELETE in Audit-Logs hervorgehoben. RPC-Stil verliert all das — jeder POST sieht gleich aus.

RPC modelliert Endpunkte oft als Funktionen oder Aktionen. REST modelliert fachliche Ressourcen und nutzt HTTP-Methoden für die Operationen auf ihnen. Dadurch kann Infrastruktur HTTP-Semantik besser auswerten, etwa für Caching, Logging, Security oder Monitoring.

Ressourcen und Repräsentationen

Eine **Ressource** ist ein fachliches Konzept. Was der Client empfaengt, ist eine **Repräsentation**.

Konzept	Bedeutung
Ressource	Das fachliche Ding (existiert im System)
URI	Die Adresse der Ressource (identifiziert sie eindeutig)
Repräsentation	Die konkrete Darstellung (JSON, HTML, CSV - je nach Accept-Header)
Zustandsübergang	Aktion auf der Ressource (GET liest, POST erzeugt, DELETE entfernt)

Content Negotiation: Client und Server einigen sich über Accept und Content-Type auf das Format.

Dieselbe Ressource kann also über dieselbe URI identifiziert werden, während ihre konkrete Darstellung je nach Client unterschiedlich ausfaellt.

Eine Ressource ist das fachliche Objekt im System, etwa ein Produkt oder eine Bestellung. Eine Repräsentation ist die konkrete Darstellung dieser Ressource, zum Beispiel JSON (JavaScript Object Notation), HTML (Hypertext Markup Language) oder CSV (Comma-Separated Values). Die URI identifiziert die Ressource; Content-Type und Accept betreffen ihre Darstellung. Genau diese Trennung erklärt auch, warum dieselbe Ressource in verschiedenen Formaten geliefert oder durch verschiedene Clients unterschiedlich verarbeitet werden kann.

Wann stößt REST an Grenzen?

Situation	REST-Problem	Alternative
Client braucht Daten aus 5 Ressourcen gleichzeitig	5 separate Requests → Latenz	GraphQL: Ein Query, ein Response
Echtzeit-Updates (Chat, Live-Ticker)	Polling ist ineffizient	WebSockets: bidirektionale Verbindung
Interne Microservice-Kommunikation	JSON-Overhead, keine Typsicherheit	gRPC: binäres Protokoll, Protobuf-Schema
Komplexe Aktionen (Batch-Operationen)	Passt nicht in CRUD	RPC-Endpunkte als Ergänzung

Kein Entweder-Oder

Viele Systeme **kombinieren** Stile: REST für öffentliche APIs, GraphQL für flexible Frontends, gRPC für interne Services. Die Wahl hängt von **Nutzungsszenarien** ab — nicht von Dogma.

REST ist kein JSON-over-HTTP. Es ist ein Architekturstil mit sechs Constraints, die das Web skalierbar, cachebar und erweiterbar machen. Die Stärke liegt in der konsequenten Nutzung bestehender Web-Infrastruktur. Wer die Constraints versteht, kann beurteilen, wann REST die richtige Wahl ist und wann ein anderer Stil besser passt.

REST passt gut für viele Web-APIs, aber nicht für alle Probleme. GraphQL eignet sich für flexible zusammengesetzte Datenabfragen, WebSockets für Echtzeitkommunikation und gRPC (Google Remote Procedure Call) für effiziente typsichere Service-zu-Service-Kommunikation. REST bleibt trotzdem oft ein robuster Default für öffentliche APIs.

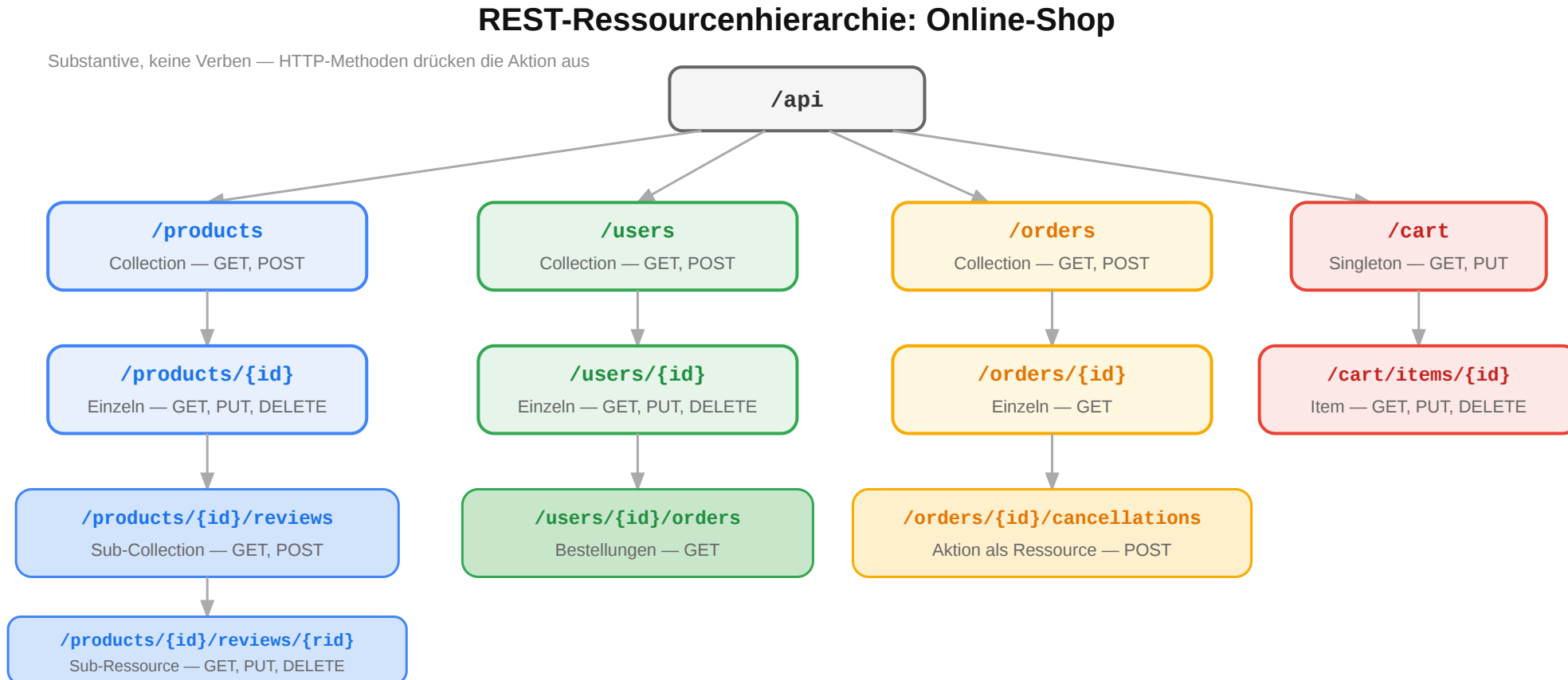
Ressourcen und URI-Design

Leitfrage: Wie werden fachliche Objekte so als Ressourcen modelliert, dass eine API verständlich und stabil bleibt?

Gute URI-Designs machen Ressourcen, Zugehörigkeit und Filter logisch sichtbar. Dabei sollte die URI fachliche Stabilität ausdrücken und nicht die interne Datenbank oder Implementierung spiegeln.

Ressourcenorientiertes Design: Online-Shop

Der erste Schritt beim API-Design ist die Identifikation der **Ressourcen** — die fachlichen Dinge, mit denen Clients interagieren:



Typische Ressourcen eines Online-Shops sind Produkte, Kategorien, Warenkoerbe, Bestellungen und Bewertungen. Zwischen Ressourcen können Hierarchien oder Beziehungen bestehen, die sich in Collections und Unterressourcen ausdruecken lassen.

URI-Design-Regeln

Regel	Gut	Schlecht	Warum
Plural für Collections	/products	/product	Collection enthält viele
Kleinschreibung, Bindestriche	/order-items	/OrderItems	URL-Konvention
Keine Verben im Pfad	POST /orders	POST /createOrder	Methode drückt Aktion aus
Hierarchie = Zugehörigkeit	/users/42/orders	/getUserOrders?userId=42	Beziehung sichtbar
Query für Filter/Sortierung	/products?sort=price	/products/sortByPrice	Pfad = Identität, Query = Parameter

Anti-Patterns erkennen

```
POST /api/getUser  
    → ?  
POST /api/deleteAccount  
    → ?  
GET /api/search?action=find → ?  
POST /api/doPayment  
    → ?
```

Ressourcenorientierte Fassung

```
POST /api/getUser  
    → GET /users/{id}  
POST /api/deleteAccount  
    → DELETE /accounts/{id}  
GET /api/search?action=find → GET  
    /products?q=...  
POST /api/doPayment  
    → POST /orders/{id}/payments
```

URIs sollten konsistent, lesbar und fachlich begründet sein. Collections stehen meist im Plural, Verben gehören nach Möglichkeit in die HTTP-Methode statt in den Pfad, und Query-Parameter dienen für Filterung, Suche oder Sortierung statt für Identität.

Übung: API-Design für eine Domäne

Aufgabe: Entwerfen Sie die REST-Ressourcen für ein **Universitäts-Kursverwaltungssystem**. Welche Ressourcen gibt es? Welche URIs? Welche Hierarchien?

Hinweis: Denken Sie an Kurse, Vorlesungen, Studierende, Einschreibungen, Noten.

Eine mögliche Lösung

Fachliches Konzept	URI	Methoden
Alle Kurse	/courses	GET, POST
Ein Kurs	/courses/web-eng	GET, PUT, DELETE
Vorlesungen eines Kurses	/courses/web-eng/lectures	GET, POST
Einschreibungen	/courses/web-eng/enrollments	GET, POST

Fachliches Konzept	URI	Methoden
Eine Einschreibung	/courses/web-eng/enrollments/42	GET, DELETE
Noten eines Studierenden	/students/42/grades	GET
Note in einem Kurs	/students/42/grades/web-eng	GET, PUT

Diskussion: Sollte die Note unter /students/42/grades/web-eng oder unter /courses/web-eng/grades/42 liegen? Beide sind valide — die Wahl hängt davon ab, **wer der primäre Nutzer der API ist**.

Beim Ressourcenentwurf werden fachliche Entitäten, Beziehungen und typische Operationen identifiziert. Mehrere URI-Strukturen können valide sein, solange sie aus Sicht der API-Nutzung konsistent und verständlich sind.

CRUD-Mapping auf HTTP

Operation	HTTP	URI	Body	Response
Liste lesen	GET	/products	—	200 + Array
Einzel lesen	GET	/products/42	—	200 + Objekt
Anlegen	POST	/products	Neues Objekt	201 + Location
Vollständig ersetzen	PUT	/products/42	Vollständig	200 / 204
Teilweise ändern	PATCH	/products/42	Nur Änderungen	200 / 204
Löschen	DELETE	/products/42	—	204

Jenseits von CRUD

Nicht alles passt in CRUD. Für **Aktionen** gibt es zwei Muster:

- **Sub-Ressource:** POST /orders/42/cancellations
- **Controller-Ressource:** POST /password-resets

CRUD (Create, Read, Update, Delete) deckt den häufigsten Fall — datenzentrierte Ressourcen — gut ab, stößt aber bei aktionsorientierten Operationen an Grenzen. Eine Bestellung stornieren ist keine DELETE-Operation auf der Bestellung selbst (die Bestellung bleibt erhalten, ihr Status ändert sich). Zwei etablierte Muster für solche Fälle: **Sub-Ressource** (POST /orders/42/cancellations — die Stornierung ist eine eigenständige Ressource, die durch POST entsteht) und **Controller-Ressource** (POST /password-resets — eine Ressource, die eine Aktion kapselt). Beide Muster bleiben HTTP-konform und vermeiden RPC-artige Verb-URLs wie /cancelOrder?id=42.

Workflow: Create / Update / Delete korrekt abbilden

Typische Zustandsänderungen und passende Responses

POST /products

201 Created + Location

PUT /products/42

200 OK oder 204

PATCH /products/42

200 OK oder 204

DELETE /products/42

204 No Content

POST erzeugt typischerweise eine neue Ressource, PUT/PATCH ändern eine vorhandene, DELETE entfernt sie.
Der Statuscode soll den fachlichen Effekt ausdrücken, nicht nur „irgendwie Erfolg“.

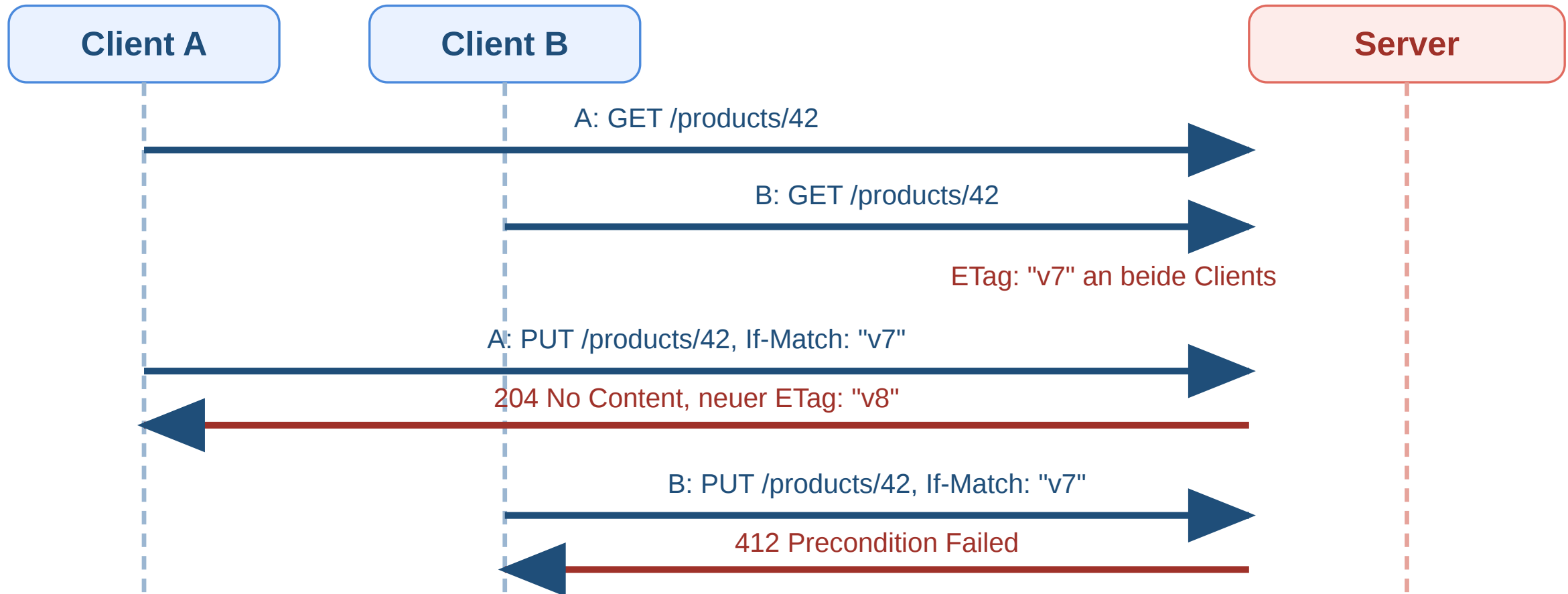
Operation	Gute Response	Warum?
Anlegen	201 Created + Location	Neue Ressource ist entstanden
Aktualisieren	200 oder 204	Ressource existierte bereits



Operation	Gute Response	Warum?
Löschen	204 No Content	Erfolgreich, aber kein Body nötig

Dieser Workflow zeigt das typische Grundmuster für zustandsändernde Ressourcenoperationen. Wichtig ist die Unterscheidung zwischen Erzeugen (201 Created) und Verändern oder Löschen (200 oder 204). Die Location-Header-Angabe ist besonders bei neu erstellten Ressourcen wertvoll. MDN: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status/201> und <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status/204>

Workflow: Conditional PUT mit ETag und If-Match



Ziel: Das PUT wird nur ausgeführt, wenn die Ressource noch die erwartete Version hat.

Das ist der typische Conditional-PUT-Workflow. Der Client sendet die neue Repräsentation nur unter der Bedingung, dass der aktuelle ETag noch zum zuletzt gelesenen Zustand passt. So wird verhindert, dass eine inzwischen veraltete Clientkopie neuere Änderungen überschreibt. If-Match formuliert diese Vorbedingung; bei Konflikt ist 412 Precondition Failed passend. MDN: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/If-Match> und <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status/412>

Pagination, Filter und Sortierung

Query-Parameter beschreiben Auswahl, Reihenfolge und Ausschnitt einer Collection — nicht die Identität der Ressource selbst.

```
GET /products?page=2&per_page=20&sort=-price&category=electronics&q=kopfhörer
```

Parameter	Funktion	Beispiel
page / per_page	Seitenbasierte Pagination	?page=3&per_page=25
cursor	Cursor-basierte Pagination	?cursor=eyJpZCI6NDJ9
sort	Sortierung (- = absteigend)	?sort=-created_at,name
filter	Filterung	?status=active&min_price=10
q	Volltextsuche	?q=kopfhörer

Pagination in der Response

```
{
  "data": [{ "id": 42, "name": "Funkkopfhörer", "price": 79.99 }],
  "meta": { "total": 142, "page": 2, "per_page": 20 },
  "links": {
    "next": "/products?page=3&per_page=20",
    "prev": "/products?page=1&per_page=20"
  }
}
```

Links machen die API navigierbar — der Client muss keine URIs konstruieren.

Gute REST-APIs beginnen mit sauber geschnittenen Ressourcen. URI-Design ist eine **Modellierungsaufgabe**: Ressourcen identifizieren, Beziehungen als Hierarchie abbilden, Aktionen über HTTP-Methoden ausdrücken. Die

Collection-Ressourcen brauchen oft zusätzliche Abfrageparameter. Pagination begrenzt die Menge der Daten, Sortierung ordnet sie, Filter schraenken sie fachlich ein. Response-Metadaten und Links helfen Clients, weitere Seiten oder Suchergebnisse systematisch zu navigieren.

Evolution, Fehler und API-Qualität

Leitfrage: Wie entwickelt man APIs weiter, ohne bestehende Clients bei jeder Änderung zu brechen — und wie misst man die Qualität einer API?

API-Qualität zeigt sich nicht nur im ersten Design, sondern auch in Wartbarkeit, Beobachtbarkeit und kontrollierter Weiterentwicklung. Additive Änderungen sind meist kompatibler als entfernende oder umbenennende Änderungen.

Welche dieser Änderungen brechen Clients?

Der Online-Shop plant ein API-Update. Bewerten Sie jede Änderung:

Geplante Änderung	Kompatibel?
Neues Feld "created_at" in Produkt-Response	
Feld "discount" aus Produkt-Response entfernen	
Neuer optionaler Query-Parameter ?include=reviews	
Feld "userName" umbenennen in "user_name"	
Neuer Endpunkt GET /products/{id}/recommendations	
Pflichtfeld "phone" in Registrierung hinzufügen	

Änderung	Kompatibel?	Begründung
Neues Feld in Response	ja	Clients ignorieren unbekannte Felder
Feld aus Response entfernen	nein	Client erwartet "discount", Feld fehlt → Fehler
Neuer optionaler Parameter	ja	Bestehende Requests funktionieren weiter
Feld umbenennen	nein	Client liest "userName", findet es nicht



Änderung	Kompatibel?	Begründung
Neuer Endpunkt	ja	Bestehende Endpunkte unverändert
Pflichtfeld hinzufügen	nein	Bestehende Clients senden "phone" nicht → 400

Prinzip: Additive Evolution

Neue Felder, Endpunkte, optionale Parameter → kompatibel. Entfernen, Umbenennen, Required machen → Breaking Change.

Rückwärtskompatibilität bedeutet, dass bestehende Clients weiter funktionieren. Neue optionale Felder oder Endpunkte sind oft unkritisch. Entfernte Felder, geänderte Namen oder neue Pflichtangaben brechen dagegen häufig vorhandene Integrationen.

API-Lebenszyklus

Phase	Herausforderung
Design	Ressourcen richtig schneiden, ohne zu viel vorwegzunehmen
Implement	Server + Client SDK konsistent halten
Test	Nicht nur „funktioniert es?“, sondern „hält der Vertrag?“
Release	Dokumentation und Versionierung synchron halten
Operate	Fehler erkennen, bevor Clients sie melden
Evolve	Neue Anforderungen einbauen, ohne Bestehendes zu brechen

Robustheitsprinzip (Postel's Law)

„Be liberal in what you accept, conservative in what you send.“

Diskussionsfrage: Wenn der Server „liberal“ ungültige Eingaben akzeptiert, entsteht ein impliziter Vertrag. Ist das wirklich eine gute Idee?

Der Lebenszyklus einer API umfasst Entwurf, Implementierung, Test, Release, Betrieb und Weiterentwicklung. In jeder Phase ist der API-Vertrag zentral: Er muss technisch korrekt, dokumentiert, beobachtbar und langfristig handhabbar bleiben.

Deprecation-Strategie

Wenn Breaking Changes unvermeidbar sind, müssen sie **angekündigt** und **schrittweise** eingeführt werden:

```
Deprecation: true  
Sunset: Sat, 01 Nov 2026 00:00:00 GMT  
Link: </docs/migration-v3>; rel="deprecation"
```

Header	Funktion
Deprecation: true	Endpunkt als veraltet markiert
Sunset	Ab diesem Datum wird entfernt
Link	Verweis auf Migrationsdokumentation

Wenn Breaking Changes unvermeidbar sind, sollten sie früh erkennbar und planbar sein. Deprecation- und Sunset-Hinweise markieren veraltete Endpunkte und geben Clients Zeit für Migration, bevor ein Endpunkt entfernt wird.

API-Dokumentation mit OpenAPI

```
openapi: 3.0.3
info:
  title: Shop API
  version: 1.0.0
paths:
  /products/{id}:
    get:
      summary: Produkt abrufen
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: integer
      responses:
        '200':
          description: Produkt gefunden
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/Product'
```

Vorteil	Erklärung
Maschinenlesbar	Automatische Code-Generierung
Interaktive Docs	Swagger UI — Entwickler können direkt testen
Contract Testing	Schema als Grundlage für automatisierte Tests



Vorteil	Erklärung
Single Source of Truth	Dokumentation und Implementierung synchron

OpenAPI beschreibt Endpunkte, Parameter, Request- und Response-Schemata maschinenlesbar. Daraus lassen sich interaktive Dokumentation, Client-Code, Validierung und Contract Tests ableiten.



Observability: APIs im Betrieb

Die vier goldenen Signale

Signal	Frage	Online-Shop-Beispiel
Latency	Wie lange dauern Requests?	p95 für GET /products < 200ms
Traffic	Wie viel Last hat das System?	5.000 Requests/s zur Cyber-Monday-Spitze
Errors	Wie hoch ist die Fehlerrate?	0,1% 5xx ist akzeptabel
Saturation	Wie nah an der Kapazitätsgrenze?	DB-Connection-Pool bei 85%

Säule	Was wird gemessen?	Werkzeuge
Logging	Einzelne Requests: Wer, Was, Wann, Ergebnis	ELK Stack, Loki
Metriken	Aggregiert: Requests/s, Latenz, Fehlerrate	Prometheus, Grafana
Tracing	Ein Request über alle Services hinweg	Jaeger, OpenTelemetry



Observability umfasst Logging, Metriken und Tracing. Die vier goldenen Signale sind Latenz, Traffic, Errors und Saturation. Zusammen helfen sie, Engpässe, Fehlerraten und Verhaltensaenderungen im Betrieb zu erkennen.

Bewerten Sie diese API

Aufgabe: Prüfen Sie die API des Online-Shops gegen diese Checkliste. Was fehlt?

```
GET /products
    → 200, kein Cache-Control
POST /products
    → 200 statt 201, kein Location
GET /products/999
    → 200 mit {"error": "not found"}
POST /orders
    → 500 bei ungültiger Eingabe
DELETE /products/42
    → 200 mit Body
Fehler-Format: mal {"error": "..."}, mal {"message": "..."}

```

Problem	Verstoß	Korrektur
Kein Cache-Control	Caching unkontrolliert	Cache-Control: no-cache + ETag
200 statt 201 bei POST	Falscher Statuscode	201 Created + Location
200 bei nicht gefunden	Statuscode lügt	404 Not Found
500 bei ungültiger Eingabe	Server-Fehler ≠ Client-Fehler	400 oder 422
Body bei DELETE	Unnötig	204 No Content
Inkonsistentes Fehlerformat	Client kann Fehler nicht parsen	Einheitliches Error-Schema



API-Design endet nicht beim ersten Release. Die eigentliche Qualität zeigt sich in **Evolution** (additive Changes, Deprecation-Strategie, Postel's Law), **Fehlerkultur** (konsistente Fehlermodelle, spezifische Statuscodes) und **Betriebsfähigkeit** (Observability mit den vier goldenen Signalen). Eine gute API ist vorhersagbar, dokumentiert, beobachtbar und entwickelt sich weiter, ohne bestehende Clients zu brechen.

Die Checkliste macht typische API-Schwächen sichtbar: unpassende Statuscodes, fehlende Cache-Regeln, inkonsistente Fehlerdarstellung und unklare Semantik. Gute APIs sind nicht nur funktional, sondern auch vorhersagbar und systematisch interpretierbar.

Wrap-Up

- **HTTP** ist das Interaktionsmodell des Webs: Methoden drücken Absichten aus, Statuscodes kommunizieren Ergebnisse, Header steuern Caching, Sessions und Sicherheit.
- **HTTP-Repräsentationen** machen sichtbar, in welchem Format eine Ressource übertragen wird und wie Clients und Server dieses Format aushandeln.
- **REST** nutzt diese HTTP-Mechanismen konsequent für APIs — Ressourcen statt Funktionsaufrufe, einheitliche Schnittstelle statt proprietärer Protokolle.
- **Gutes API-Design** beginnt bei sauber geschnittenen Ressourcen, nutzt HTTP-Semantik vollständig und plant Evolution von Anfang an mit.
- **Betrieb** erfordert Observability, Rate Limiting und konsistente Fehlermodelle.

Die nächste Vorlesung zeigt, wie diese APIs **serverseitig implementiert** werden — am Beispiel des Java-Web-Stacks.

HTTP liefert die grundlegende Semantik für Kommunikation im Web: Methoden, Statuscodes, Header und Repräsentationen. REST nutzt genau diese Semantik für Ressourcen und ihre Darstellungen. Gute APIs kombinieren korrekt gewählte Methoden, Statuscodes, Header, konsistente Fehlermodelle und einen planbaren Evolutionspfad.

